PART 4 OF 5 · SOLUTIONS

Stochastic Calculus, Step by Step

Part 4: pricing and numerics.
Solutions to every exercise



Rates with Bip

STOCHASTIC CALCULUS, STEP BY STEP · PART 4 OF 5 · SOLUTIONS

Stochastic Calculus, Step by Step

Part 4: Continuous-time pricing and numerical methods. Solutions

Rates with Bip

Stochastic Calculus, Step by Step. Part 4: Continuous-time pricing and numerical methods. Solutions.

© 2026 Rates with Bip. This book is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International licence (CC BY-NC-ND 4.0): you may copy and share it unchanged, for non-commercial purposes, crediting Rates with Bip. Licence: creativecommons.org/licenses/by-nc-nd/4.0. The companion notebooks are free under the MIT licence at github.com/RatesWithBip/curriculum.

The course's numbers: $S_0 = 100$, $K = 105$, $r = 3\%$ (continuously compounded), $\sigma = 20\%$, $T = 1$ year, and $\mu = 8\%$ where a real-world return is needed. Made up for teaching; every random number is simulated with a fixed seed.

This volume holds the full solutions to the exercises and self-test questions in Stochastic Calculus, Step by Step, Part 4. Try each exercise before reading its solution.

Every number in the solutions is computed by a script that checks itself with assertions; the code solutions are shown in full.

This book is for education only. It is not financial, investment, legal or tax advice.

Contents

How to use the solutions 5

19. The Black-Scholes Equation 6

20. The Black-Scholes Formula 12

21. The Greeks 16

22. Checking the Formula Numerically 22

23. The Trinomial Tree 27

24. Delta Hedging in Practice 32

25. Girsanov's Theorem 38

26. Risk-Neutral Pricing 43

27. Change of Numéraire 48

28. The Feynman-Kac Theorem 53

29. Finite Differences 58

30. Choosing a Method 64

How to use the solutions

This volume holds the full solutions to *Stochastic Calculus, Step by Step, Part 4: Continuous-time pricing and numerical methods*: every exercise worked out, and the answers to the self-test questions at the end of each lesson. It is meant to be read **after** you have tried each one.

- **By-hand exercises** (the first six of each lesson) are worked in full. Each solution starts with **the idea** (the one thought that unlocks it), then goes through numbered steps with the reason for each, and ends with a check or an interpretation. If you are stuck, read only the idea and try again before reading the steps.
- **Code exercises** (the last two of each lesson) show a complete Python solution and exactly what it prints. Every number in this volume is produced by a script that checks itself with assertions, using fixed random seeds, so your own run should match to the digits shown when you use the same seeds.
- **Self-test answers** are short: a sentence or two, and a number where there is one.

The course's numbers are the same throughout: $S_0 = 100$, $K = 105$, $r = 3\%$ (continuously compounded), $\sigma = 20\%$, $T = 1$ year, and $\mu = 8\%$, so that $\theta = (\mu - r)/\sigma = 0.25$. The call is worth 7.1281 and the put 9.0248.

Almost every by-hand solution in Part 4 uses one of three moves: **match** the dW and dt terms of two equal expressions; **complete the square** in a normal integral, which turns a weight into a shift of the bell curve; or **use a martingale**, whose dt term is zero and whose average stays put. If your answer differs, check the usual suspects: μ written where r belongs, V_t confused with V_τ, a forgotten discount on the strike, a missing $\sqrt{T}$, and a standard deviation written where a variance belongs.

This volume is for education only and is not financial advice.

SOLUTIONS TO LESSON 19

The Black-Scholes Equation

Solutions to Exercises 19.1 to 19.8 and the Lesson 19 self-test.

Exercise 19.1

The idea. Count the two sources of gain, then discount with the product rule.

Step 1. The rule. Over dt, the Δ_t shares gain $\Delta_t\,dS_t$. The cash is what's left, $X_t - \Delta_t S_t$, and it earns $r(X_t - \Delta_t S_t)\,dt$. Nothing else enters or leaves, so $dX_t = \Delta_t\,dS_t + r(X_t - \Delta_t S_t)\,dt$.

Step 2. The product rule on $e^{-rt}X_t$. The discount factor has no dW part, so there's no cross-term:

$$d(e^{-rt}X_t) = e^{-rt}\,dX_t - re^{-rt}X_t\,dt.$$

Step 3. Substitute Step 1. The terms in rX_t cancel:

$$d(e^{-rt}X_t) = e^{-rt}\big(\Delta_t\,dS_t - r\Delta_t S_t\,dt\big).$$

Step 4. Recognise. By the same product rule, $d(e^{-rt}S_t) = e^{-rt}(dS_t - rS_t\,dt)$. So $d(e^{-rt}X_t) = \Delta_t\,d(e^{-rt}S_t)$.

In words. Measured in today's money, your wealth changes only through your position in the discounted share. The cash, once discounted, is flat.

Exercise 19.2

The idea. Taylor to second order, then the multiplication table of Lesson 14.

Step 1. Taylor.

$$dV = V_t\,dt + V_S\,dS + \tfrac{1}{2}V_{SS}(dS)^2 + V_{tS}\,dt\,dS + \tfrac{1}{2}V_{tt}(dt)^2 + \dots$$

Step 2. The squares. With $dS = \mu S\,dt + \sigma S\,dW$ and the table $dW\,dW = dt$, $dW\,dt = 0$, $dt\,dt = 0$:

$$(dS)^2 = \sigma^2 S^2\,dt, \qquad dt\,dS = 0, \qquad (dt)^2 = 0.$$

Higher terms are smaller still, so they vanish too.

Step 3. Collect.

$$dV = \left(V_t + \mu S V_S + \tfrac{1}{2}\sigma^2 S^2 V_{SS}\right) dt + \sigma S V_S \, dW.$$

Exercise 19.3

The idea. Hold V_S shares short against the option, and the random part vanishes. A riskless position must earn r.

Step 1. Over the step, with V_S held fixed, $d\Pi = dV - V_S \, dS$.

Step 2. Substitute Itô's lemma and dS.

$$d\Pi = \left(V_t + \mu S V_S + \tfrac{1}{2}\sigma^2 S^2 V_{SS}\right) dt + \sigma S V_S \, dW - V_S(\mu S \, dt + \sigma S \, dW).$$

The dW terms cancel, and so do the $\mu S V_S \, dt$ terms:

$$d\Pi = \left(V_t + \tfrac{1}{2}\sigma^2 S^2 V_{SS}\right) dt.$$

Step 3. No risk, so the cash rate. If Π earned more than r, you'd borrow to hold it; if less, you'd short it and invest in cash. Either way there's free money, so $d\Pi = r\Pi \, dt = r(V - V_S S) \, dt$.

Step 4. Equate. $V_t + \frac{1}{2}\sigma^2 S^2 V_{SS} = rV - rSV_S$, which rearranges to the Black-Scholes equation. ✓

Exercise 19.4

The idea. Direct substitution, and the equation is linear.

$V = S$**.** $V_t = 0$, $V_S = 1$, $V_{SS} = 0$. Left side: rS. Right side: rS. ✓

$V = Ke^{-r(T-t)}$**.** $V_t = rKe^{-r(T-t)} = rV$; $V_S = V_{SS} = 0$. Left side: rV. Right side: rV. ✓

Their difference. $S - Ke^{-r(T-t)}$ is worth $S_T - K$ at T: a **forward contract** to buy at K. With our numbers it's worth -1.8968 today, negative because the strike is above the forward price.

Why it must solve the equation. The equation is linear: every term is a derivative of V times a coefficient that doesn't depend on V. So a sum or difference of solutions is a solution. Financially, a portfolio of hedged, riskless positions is hedged and riskless.

Exercise 19.5

The idea. Substitute and read off the constant.

Step 1. The derivatives. $V = S^2 e^{b(T-t)}$: $V_t = -bV$, $V_S = 2Se^{b(T-t)}$, $V_{SS} = 2e^{b(T-t)}$. So $SV_S = 2V$ and $S^2 V_{SS} = 2V$.

Step 2. Substitute. $-bV + 2rV + \sigma^2 V = rV$.

Step 3. Divide by V. $b = r + \sigma^2 = 0.07$.

At $t = T$, $V = S_T^2$, the payoff. ✓ So the contract is worth

$$S_0^2 e^{(r+\sigma^2)T} = 10{,}000\, e^{0.07} = 10725.08.$$

A check. As a risk-neutral average (Lesson 17 with $\mu = r$): $e^{-rT} E[S_T^2] = e^{-rT} S_0^2 e^{(2r+\sigma^2)T}$, which is 10725.08. ✓

Exercise 19.6

The idea. A dividend is extra income on the shares, in the self-financing rule. Nothing else changes.

Step 1. The rule. Now $dX = \Delta\, dS + \Delta q S\, dt + r(X - \Delta S)\, dt$. The share's own price still follows $dS = \mu S\, dt + \sigma S\, dW$.

Step 2. Substitute dS.

$$dX = \big(rX + \Delta(\mu + q - r)S\big)\, dt + \Delta\sigma S\, dW.$$

Step 3. Match dW with Itô's lemma: $\Delta = V_S$, as before.

Step 4. Match dt. $rV + V_S(\mu + q - r)S = V_t + \mu S V_S + \frac{1}{2}\sigma^2 S^2 V_{SS}$. The $\mu S V_S$ cancels. Rearranging:

$$V_t + (r - q)SV_S + \tfrac{1}{2}\sigma^2 S^2 V_{SS} = rV.$$

In words. The share's growth in the equation drops to $r - q$, since part of its return comes as cash.

Exercise 19.7

The equation's residual, for the call, the put and the payoff itself.

```
import numpy as np

Ncdf = np.vectorize(lambda x: 0.5 * (1 + math.erf(x / math.sqrt(2))))


def call(s, tau):
    dp = (np.log(s / K) + (r + 0.5 * sigma**2) * tau) / (sigma * np.sqrt(tau))
    return s * Ncdf(dp) - K * np.exp(-r * tau) * Ncdf(dp - sigma * np.sqrt(tau))


put = lambda s, tau: call(s, tau) - s + K * np.exp(-r * tau)      # put-call parity


def terms(f, s, tau, h=0.01, k=1e-5):
    Vt = (f(s, tau - k) - f(s, tau + k)) / (2 * k)                # t up means tau down
    Vs = (f(s + h, tau) - f(s - h, tau)) / (2 * h)
    Vss = (f(s + h, tau) - 2 * f(s, tau) + f(s - h, tau)) / h**2
    return Vt, r * s * Vs, 0.5 * sigma**2 * s**2 * Vss, r * f(s, tau)


def residual(f, s, tau):
    Vt, a, b, c = terms(f, s, tau)
    return Vt + a + b - c


worst = {"call": 0.0, "put": 0.0}
for s in (70.0, 85.0, 100.0, 115.0, 130.0):
    for tau in (0.25, 0.5, 1.0):
        worst["call"] = max(worst["call"], abs(residual(call, s, tau)))
        worst["put"] = max(worst["put"], abs(residual(put, s, tau)))
print(f"largest residual: call {worst['call']:.1e}, put {worst['put']:.1e}")

payoff = lambda s, tau: max(s - K, 0.0)
print("payoff shape:      V_t   rSV_S  (1/2)s^2S^2V_SS     rV   residual")
for s in (100.0, 110.0):
    Vt, a, b, c = terms(payoff, s, 0.5)
    print(f"  S = {s:5.1f}   {Vt:6.2f}  {a:6.2f}  {b:15.2f}  {c:5.2f}  {Vt + a + b -
c:9.2f}")
```

What it prints.

```
largest residual: call 5.5e-07, put 5.4e-07
payoff shape:      V_t   rSV_S  (1/2)s^2S^2V_SS     rV   residual
  S = 100.0     0.00    0.00            0.00   0.00       0.00
  S = 110.0     0.00    3.30            0.00   0.15       3.15
```

Reading it. The call and put both satisfy the equation, up to the rounding of the bumps. The payoff-shaped function doesn't. At $S = 110$ it has $V_S = 1$ and no time dependence, so the left side is rS while the right is $r(S - K)$. The gap is exactly $rK = 3.15$: the term V_t is missing. At $S = 100$ every term is zero, and the equation holds, but the func-

tion has a kink at the strike, where V_{SS} doesn't exist. A real price must fall smoothly from the region above the strike into the region below, and V_t is what makes that possible.

Exercise 19.8

The hedged change at three step sizes.

```
rng = np.random.default_rng(198)
Z = rng.standard_normal(400_000)
V0 = float(call(S0, T))
delta = float((call(S0 + 0.01, T) - call(S0 - 0.01, T)) / 0.02)
print(" h (days)   sd of hedged change")
prev, ratios = None, []
for days in (16, 4, 1):
    h = days / 252
    S1 = S0 * np.exp((mu - 0.5 * sigma**2) * h + sigma * math.sqrt(h) * Z)
    change = call(S1, T - h) - V0 - delta * (S1 - S0)
    sd = change.std()
    ratios.append(float(prev / sd) if prev else 0.0)
    prev = sd
    print(f"{days:9d}   {sd:19.5f}")
print("shrink factors:", [round(x, 2) for x in ratios[1:]])
```

What it prints.

```
 h (days)   sd of hedged change
       16               0.36348
        4               0.08990
        1               0.02241
shrink factors: [4.04, 4.01]
```

Reading it. The spread shrinks by about 4 each time h is divided by 4: it's proportional to h. The reason is the remainder of the Taylor expansion: after the hedge removes $V_S\,\delta S$, what's left is mainly $\frac{1}{2}V_{SS}(\delta S)^2$, which is proportional to h times a random Z^2. Compare the option alone, whose spread is proportional to $\sqrt{h}$. The ratio of the two goes to zero as h goes to 0, which is why the hedge is exact in the limit.

Self-test

1. V_S: the slope of the option's value against the share.

2. Because it cancels. It appears in the option's drift ($\mu S V_S$), and identically in the hedge's drift, and the two are matched.

3. Time decay, $V_t = $ -5.28, and the convexity term, $\frac{1}{2}\sigma^2 S^2 V_{SS} = 3.99$. Their sum with rSV_S must equal the small number rV, because a hedged position earns only the cash rate: what it earns from convexity, it must pay in time decay.

SOLUTIONS TO LESSON 20

The Black-Scholes Formula

Solutions to Exercises 20.1 to 20.8 and the Lesson 20 self-test.

Exercise 20.1

The idea. Divide, take logs, solve for Z.

Step 1. $S_0 e^{(r-\frac{1}{2}\sigma^2)T+\sigma\sqrt{T}Z} > K \iff (r-\frac{1}{2}\sigma^2)T+\sigma\sqrt{T}Z > \ln(K/S_0)$. (Logs keep the inequality, since ln is increasing.)

Step 2. Using $\ln(K/S_0) = -\ln(S_0/K)$, and dividing by $\sigma\sqrt{T} > 0$:

$$Z > -\frac{\ln(S_0/K)+(r-\frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}} = -d_-.$$

In numbers. The numerator is $\ln(100/105)+0.01 = -0.04879+0.01 = -0.03879$, and the denominator is 0.2. So d_- = -0.19395, and the critical Z is 0.19395.

Exercise 20.2

The idea. Just the formula, with the discounted strike computed first.

- **Share term:** $S_0 N(d_+) = 100 \times 0.50241 = 50.241$.
- **Discounted strike:** $105e^{-0.03} = 101.897$.
- **Strike term:** $101.897 \times 0.42311 = 43.114$.
- **Call:** $50.241 - 43.114 = 7.127$.

The first term is the money in shares in the hedge. The second is what's borrowed to pay for it.

Exercise 20.3

The idea. A normal density times an exponential is a shifted normal density.

Step 1. The integral. With $a = \sigma\sqrt{T}$, over $z > -d_-$:

$$e^{-rT}\tilde{E}[S_T\mathbf{1}\{S_T>K\}] = e^{-rT}\int_{-d_-}^{\infty} S_0 e^{(r-\frac{1}{2}\sigma^2)T+az}\varphi(z)\,dz = S_0 e^{-\frac{1}{2}a^2}\int_{-d_-}^{\infty} e^{az}\varphi(z)\,dz.$$

Step 2. Complete the square. $az-\frac{1}{2}z^2 = -\frac{1}{2}(z-a)^2+\frac{1}{2}a^2$, so $e^{az}\varphi(z) = e^{\frac{1}{2}a^2}\varphi(z-a)$.

Step 3. Cancel. The two constants are $e^{-\frac{1}{2}a^2}$ and $e^{\frac{1}{2}a^2}$:

$$= S_0\int_{-d_-}^{\infty}\varphi(z-a)\,dz.$$

Step 4. Substitute $u=z-a$:

$$= S_0\int_{-d_--a}^{\infty}\varphi(u)\,du = S_0P(Z>-d_--a) = S_0N(d_-+a) = S_0N(d_+).$$

Exercise 20.4

The idea. The put pays when $Z<-d_-$. Use the same two terms, on the other side.

Step 1. Split. $P = Ke^{-rT}\tilde{P}(S_T<K) - e^{-rT}\tilde{E}[S_T\mathbf{1}\{S_T<K\}]$.

Step 2. The strike term. $\tilde{P}(S_T<K) = P(Z<-d_-) = N(-d_-)$.

Step 3. The share term. As in Exercise 20.3, but integrating from $-\infty$ to $-d_-$:

$$S_0\int_{-\infty}^{-d_-}\varphi(z-a)\,dz = S_0\int_{-\infty}^{-d_--a}\varphi(u)\,du = S_0N(-d_+).$$

So $P = Ke^{-rT}N(-d_-) - S_0N(-d_+) = 9.0248$.

Parity. $N(-x) = 1-N(x)$, so

$$P = Ke^{-rT}(1-N(d_-)) - S_0(1-N(d_+)) = C - S_0 + Ke^{-rT}.$$

✓ In numbers, $C-S_0+Ke^{-rT} = 9.0242$ (using the rounded values of Exercise 20.2).

Exercise 20.5

The idea. These are the two terms of the formula, sold separately.

- **Cash digital:** $e^{-rT}\tilde{P}(S_T>K) = e^{-rT}N(d_-) = 0.4106$.
- **Asset digital:** $e^{-rT}\tilde{E}[S_T\mathbf{1}\{S_T>K\}] = S_0N(d_+) = 50.2413$.

The call. If $S_T>K$: an asset digital pays S_T, and K cash digitals pay K, so the difference pays S_T-K. Otherwise everything pays 0. So the call is an asset digital minus K cash digitals: $50.2413 - 105\times 0.4106 = 7.1281$. ✓

Exercise 20.6

The idea. Watch the sign of $\ln(S_0/K) + rT$ in $d_\pm$, since the denominator goes to 0.

(a) $\sigma \to 0$**.** $d_\pm \to \pm\infty$, according to the sign of $S_0 - Ke^{-rT}$. If positive, both N's go to 1, and $C \to S_0 - Ke^{-rT}$. If negative, both go to 0, and $C \to 0$. So $C \to \max(S_0 - Ke^{-rT}, 0)$. With no randomness, the share grows at r, and you know the outcome today. With our numbers, $Ke^{-rT} = 101.90$, above the share, so the limit is 0.

(b) $T \to 0$**.** Similarly, $C \to \max(S_0 - K, 0)$: the payoff itself.

(c) $S_0 \to \infty$**.** Both $d_\pm \to \infty$, so $C \to S_0 - Ke^{-rT}$. The call is sure to be exercised, and so it's worth the same as a forward, a share minus a discounted strike.

Exercise 20.7

The trapezium rule on payoff times density. The columns are errors against the formula.

```
import numpy as np


def formula(k):
    d2 = (math.log(S0 / k) + (r - 0.5 * sigma**2) * T) / (sigma * math.sqrt(T))
    return S0 * N(d2 + sigma * math.sqrt(T)) - k * disc * N(d2)


def integrate(k, points):
    # Trapezium rule on payoff times density, z from -8 to 8.
    z = np.linspace(-8, 8, points)
    s_t = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * z)
    y = disc * np.maximum(s_t - k, 0) * np.exp(-z**2 / 2) / math.sqrt(2 * math.pi)
    return float(((y[1:] + y[:-1]) / 2 * np.diff(z)).sum())


print(" strike   formula      100 pts     1000 pts    10000 pts")
for k in (85, 95, 105, 115, 125):
    print(f"{k:7d}  {formula(k):8.4f}  " + "  ".join(f"{integrate(k, p) -
formula(k):+10.6f}" for p in (100, 1000, 10000)))
```

What it prints.

```
 strike   formula      100 pts     1000 pts    10000 pts
     85   19.1313   -0.001521   +0.000037   +0.000000
     95   12.1797   +0.006564   +0.000059   -0.000001
    105    7.1281   +0.004564   +0.000073   +0.000000
    115    3.8596   +0.007865   -0.000145   -0.000001
    125    1.9521   -0.006090   -0.000090   +0.000001
```

Reading it. With 1000 points on $[-8, 8]$, a spacing of 0.016, the worst error is 1.5e-04, good to about 3 or 4 decimals, depending on the strike. With 10,000 points, it's good to about 6. The errors are not perfectly steady, for the same reason as a tree's: the kink at $S_T = K$ falls at a different place among the grid's points for each strike. Putting a grid point at the kink, and integrating from there as the notebook does, removes this.

Exercise 20.8

Digital and call by Monte Carlo, with 95% half-widths.

```
rng = np.random.default_rng(208)
M = 1_000_000
Z = rng.standard_normal(M)
S_T = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * Z)
print("           estimate   95% half-width   exact      half-width/price")
for name, y, exact in (("digital", disc * (S_T > K), disc * N(dm)), ("call", disc *
np.maximum(S_T - K, 0), S0 * N(dp) - K * disc * N(dm))):
    est, hw = y.mean(), 1.96 * y.std(ddof=1) / math.sqrt(M)
    print(f"{name:9s} {est:9.5f}   {hw:14.5f}   {exact:7.5f}   {hw / est:12.3%}")
```

What it prints.

```
           estimate   95% half-width   exact      half-width/price
digital     0.41124          0.00094   0.41060         0.229%
call        7.12716          0.02457   7.12806         0.345%
```

Reading it. The call's half-width is larger, both in absolute terms and relative to its price. The relative error is set by the standard deviation of one payoff over its mean: 1.17 for the digital, and 1.76 for the call. A digital pays at most 1, so its payoff has a limited spread. A call has a long right tail: rare, very large payoffs drive most of its variance.

Self-test

1. $N(d_-) = 0.4231$, about 42%.

2. It moves the bell curve to the right by $\sigma\sqrt{T}$, without changing its shape (apart from a constant factor).

3. 50.24 and 43.11. Together they're the hedge: about half a share, financed by a loan.

SOLUTIONS TO LESSON 21

The Greeks

Solutions to Exercises 21.1 to 21.8 and the Lesson 21 self-test.

Exercise 21.1

The idea. Express $\varphi(d_+)$ through $\varphi(d_-)$, using the definition of d_-.

Step 1. $d_+^2 = (d_- + \sigma\sqrt{\tau})^2 = d_-^2 + 2d_-\sigma\sqrt{\tau} + \sigma^2\tau$.

Step 2. So $\varphi(d_+) = \varphi(d_-)\exp\big(-d_-\sigma\sqrt{\tau} - \tfrac{1}{2}\sigma^2\tau\big)$.

Step 3. From the definition of d_-: $d_-\sigma\sqrt{\tau} = \ln(S/K) + r\tau - \tfrac{1}{2}\sigma^2\tau$. Substituting, the $\tfrac{1}{2}\sigma^2\tau$ terms cancel:

$$\varphi(d_+) = \varphi(d_-)\exp\big(-\ln(S/K) - r\tau\big) = \varphi(d_-)\frac{K}{S}e^{-r\tau}.$$

Step 4. Multiply by S: $S\varphi(d_+) = Ke^{-r\tau}\varphi(d_-)$. ■

Exercise 21.2

The idea. The product rule, the chain rule, and then the identity makes the extras vanish.

Step 1. The derivative of $d_\pm$. Only $\ln S$ depends on S, so $\dfrac{\partial d_\pm}{\partial S} = \dfrac{1}{S\sigma\sqrt{\tau}}$.

Step 2. Differentiate. Using $N' = \varphi$:

$$\frac{\partial C}{\partial S} = N(d_+) + S\varphi(d_+)\frac{1}{S\sigma\sqrt{\tau}} - Ke^{-r\tau}\varphi(d_-)\frac{1}{S\sigma\sqrt{\tau}}.$$

Step 3. Use the identity. The last two terms are equal and opposite. So $\Delta = N(d_+)$.

Exercise 21.3

The idea. One more chain rule.

$$\Gamma = \frac{\partial}{\partial S}N(d_+) = \varphi(d_+)\frac{\partial d_+}{\partial S} = \frac{\varphi(d_+)}{S\sigma\sqrt{\tau}}.$$

At $S = 100, \tau = 1$. $\varphi(d_+) = 0.39893$, so $\Gamma = 0.39893/(100 \times 0.2) = 0.01995$.

The effect. A rise of 1 in the share raises delta by about $\Gamma \times 1 = 0.020$, from 0.502 to about 0.522.

Exercise 21.4

The idea. Differentiate, using the identity on the $\partial d_\pm/\partial t$ pieces. What's left is two terms.

Step 1. With $\tau = T - t$, the discount factor gives $\dfrac{\partial}{\partial t} e^{-r\tau} = re^{-r\tau}$. So

$$\frac{\partial C}{\partial t} = S\varphi(d_+)\frac{\partial d_+}{\partial t} - Ke^{-r\tau}\varphi(d_-)\frac{\partial d_-}{\partial t} - rKe^{-r\tau}N(d_-).$$

Step 2. The identity. It replaces $Ke^{-r\tau}\varphi(d_-)$ by $S\varphi(d_+)$, so the first two terms become $S\varphi(d_+)\dfrac{\partial}{\partial t}(d_+ - d_-)$.

Step 3. The difference. $d_+ - d_- = \sigma\sqrt{\tau}$, so $\dfrac{\partial}{\partial t}(\sigma\sqrt{T-t}) = -\dfrac{\sigma}{2\sqrt{\tau}}$. Hence

$$\Theta = -\frac{S\varphi(d_+)\sigma}{2\sqrt{\tau}} - rKe^{-r\tau}N(d_-).$$

Step 4. The equation. Use $\frac{1}{2}\sigma^2 S^2\Gamma = \frac{1}{2}\sigma^2 S^2 \dfrac{\varphi(d_+)}{S\sigma\sqrt{\tau}} = \dfrac{S\varphi(d_+)\sigma}{2\sqrt{\tau}}$. So the first term of Θ cancels the Γ term:

$$\begin{aligned}\Theta + rS\Delta + \tfrac{1}{2}\sigma^2 S^2\Gamma &= -rKe^{-r\tau}N(d_-) + rSN(d_+) \\ &= r\big(SN(d_+) - Ke^{-r\tau}N(d_-)\big) = rC.\end{aligned}$$

✓

Exercise 21.5

The idea. $P = C - S + Ke^{-r\tau}$, so differentiate the three terms.

- **Delta:** $\Delta_P = \Delta_C - 1 =$ -0.4976.
- **Gamma:** the terms $-S$ and $Ke^{-r\tau}$ are straight lines in S, with no second derivative. So $\Gamma_P = \Gamma_C = 0.01995$.
- **Theta:** $\Theta_P = \Theta_C + rKe^{-r\tau} =$ -5.2827 + 3.0569 = -2.2258.

Which are the same, and why. Gamma, and vega too: the difference between a call and a put is a forward, whose value is linear in S and doesn't depend on σ. Both options carry the same convexity. Theta differs, since the put holds a claim on the discounted strike,

which grows with time.

Exercise 21.6

The idea. Same pattern again: only the σ inside $d_{\pm}$ matters, and the identity collapses it.

Step 1. Both d_+ and d_- depend on σ, and $d_+ - d_- = \sigma\sqrt{\tau}$.

$$\frac{\partial C}{\partial \sigma} = S\varphi(d_+)\frac{\partial d_+}{\partial \sigma} - Ke^{-r\tau}\varphi(d_-)\frac{\partial d_-}{\partial \sigma} = S\varphi(d_+)\frac{\partial (d_+ - d_-)}{\partial \sigma} = S\varphi(d_+)\sqrt{\tau}.$$

Step 2. Using $\Gamma = \varphi(d_+)/(S\sigma\sqrt{\tau})$, we get $\varphi(d_+) = \Gamma S\sigma\sqrt{\tau}$, so vega is $S \cdot \Gamma S\sigma\sqrt{\tau} \cdot \sqrt{\tau} = \sigma\tau S^2\Gamma$.

In numbers. 39.89 both ways. A one-point rise in volatility adds 0.40.

Why so closely related. Volatility enters the price only through the expected size of squared moves, $\sigma^2\tau$. And it's gamma that turns squared moves into money. More volatility and more convexity do the same work.

Exercise 21.7

Bumps against formulas, then the size of the gamma error against the bump.

```
import numpy as np

phi = lambda x: math.exp(-x**2 / 2) / math.sqrt(2 * math.pi)


def d1(s, tau, sig=sigma):
    return (math.log(s / K) + (r + 0.5 * sig**2) * tau) / (sig * math.sqrt(tau))


def call(s, tau, sig=sigma):
    dp = d1(s, tau, sig)
    return s * Nf(dp) - K * math.exp(-r * tau) * Nf(dp - sig * math.sqrt(tau))


def formulas(s, tau=T):
    dp = d1(s, tau); dm = dp - sigma * math.sqrt(tau)
    return {"delta": Nf(dp), "gamma": phi(dp) / (s * sigma * math.sqrt(tau)),
            "theta": -s * phi(dp) * sigma / (2 * math.sqrt(tau)) - r * K * math.exp(-r *
tau) * Nf(dm), "vega": s * phi(dp) * math.sqrt(tau)}


def bumps(s, tau=T, h=0.01, k=1e-5, v=1e-4):
    return {"delta": (call(s + h, tau) - call(s - h, tau)) / (2 * h), "gamma": (call(s + h,
tau) - 2 * call(s, tau) + call(s - h, tau)) / h**2,
            "theta": (call(s, tau - k) - call(s, tau + k)) / (2 * k), "vega": (call(s, tau,
sigma + v) - call(s, tau, sigma - v)) / (2 * v)}


print("   S   greek      formula         bump")
for s in (80.0, 100.0, 120.0):
    f_, b_ = formulas(s), bumps(s)
    for g in ("delta", "gamma", "theta", "vega"):
        print(f"{s:4.0f}   {g:6s} {f_[g]:11.6f}  {b_[g]:11.6f}")
print("gamma error at S = 100, by bump size:")
exact = formulas(100.0)["gamma"]
errs = {}
for h in (1.0, 0.1, 0.01, 0.001, 0.0001):
    errs[h] = bumps(100.0, h=h)["gamma"] - exact
    print(f"  h = {h:<7g} {errs[h]:+.2e}")
```

What it prints.

```
   S   greek      formula         bump
  80   delta     0.133571     0.133571
  80   gamma     0.013471     0.013471
  80   theta    -2.015176    -2.015176
  80   vega     17.242994    17.242993
 100   delta     0.502413     0.502413
 100   gamma     0.019947     0.019947
 100   theta    -5.282748    -5.282748
 100   vega     39.893498    39.893498
 120   delta     0.820601     0.820601
 120   gamma     0.010910     0.010910
 120   theta    -5.476186    -5.476186
 120   vega     31.421930    31.421928
gamma error at S = 100, by bump size:
  h = 1       -3.81e-06
  h = 0.1     -3.81e-08
  h = 0.01    -3.86e-10
  h = 0.001   -2.59e-09
  h = 0.0001  +3.17e-07
```

Reading it. The bumps match the formulas. The gamma error falls by 100 each time the bump is cut by 10, which is the h^2 error of Lesson 29, until about $h = 0.01$. Then it rises again. That's rounding: the second difference divides a tiny number, known to about 10^{-14}, by h^2. So the error grows like $10^{-14}/h^2$ for small h. There's a best bump, and for second derivatives it isn't very small.

Exercise 21.8

The exact one-day profit of a hedged long call, against the gamma formula.

```python
dt = 1 / 252
V, D, G = call(100.0, T), formulas(100.0)["delta"], formulas(100.0)["gamma"]
print("  move     exact P&L    gamma formula")
pl = {}
for m in (0.0, 0.005, -0.005, 0.0126, -0.0126, 0.025, -0.025):
    s1 = 100.0 * (1 + m)
    exact_pl = call(s1, T - dt) - V - D * (s1 - 100.0) - r * (V - D * 100.0) * dt
    approx = 0.5 * G * 100.0**2 * (m**2 - sigma**2 * dt)
    pl[m] = exact_pl
    print(f"{m:+7.2%}   {exact_pl:9.4f}   {approx:12.4f}")
print(f"break-even move: sigma sqrt(dt) = {sigma * math.sqrt(dt):.2%}")
```

What it prints.

```
   move     exact P&L    gamma formula
 +0.00%     -0.0158        -0.0158
 +0.50%     -0.0135        -0.0133
 -0.50%     -0.0131        -0.0133
 +1.26%     -0.0005         0.0000
 -1.26%      0.0006         0.0000
 +2.50%      0.0450         0.0465
 -2.50%      0.0480         0.0465
break-even move: sigma sqrt(dt) = 1.26%
```

Reading it. The formula is close, and improves as the move shrinks. An unchanged share costs the holder 0.016 a day. Moves of about $\pm 1.26\%$ break even. Larger moves, of either sign, pay. The small differences between up and down moves are the third-order term that the formula leaves out.

Self-test

1. $\Delta = 0.5024, \Gamma = 0.01995$.

2. A call minus a put is a forward, whose value is a straight line in S. A straight line has no curvature, so both options have the same gamma.

3. They lose. Theta is negative: with no move, gamma earns nothing, and there's the waiting to pay for.

SOLUTIONS TO LESSON 22

Checking the Formula Numerically

Solutions to Exercises 22.1 to 22.8 and the Lesson 22 self-test.

Exercise 22.1

The idea. Solve $1.96s/\sqrt{M} = h$ for M.

$M = (1.96s/h)^2$. With $s = 12.5$:

- $h = 0.01$: $M = 6{,}002{,}500$ paths.
- $h = 0.001$: $M = 600{,}250{,}000$ paths.

Why 100, not 10. The error falls like $1/\sqrt{M}$, so M grows like $1/h^2$. A tenth of the error costs a hundred times the paths.

Exercise 22.2

The idea. The variance of an average of two correlated quantities.

Step 1. Z and $-Z$ have the same distribution, so $f(Z)$ and $f(-Z)$ have the same variance, v. Their covariance is ρv.

Step 2.

$$\mathrm{Var}\Big(\tfrac{1}{2}\big(f(Z) + f(-Z)\big)\Big) = \tfrac{1}{4}(v + v + 2\rho v) = \tfrac{1}{2}v(1 + \rho).$$

Why this helps. One pair uses two payoffs, so compare with the average of two independent draws, whose variance is $\frac{1}{2}v$. Antithetic is better when $\rho < 0$. For a monotone payoff, $f(Z)$ and $f(-Z)$ move in opposite directions: when one is high, the other is low. So $\rho < 0$ always, and the method can't hurt. (For a payoff like a straddle, which is high at both extremes, ρ can be positive, and it does hurt.)

Exercise 22.3

The idea. The kink falls between nodes, and where it falls keeps changing.

Calls. The final nodes are spaced by a factor u^2. A tree effectively replaces the kinked payoff by something depending on where K sits among them. As n changes by one, all the nodes move, so K's position within its gap changes, and the rounding of the kink changes with it. The error therefore jumps up and down, and only its envelope shrinks like $1/n$.

Puts. They have the same kink at the same K. In fact, on the same tree, put-call parity holds exactly for u, d and $\tilde{p}$ chosen so that the discounted share is a martingale: the call and put errors are identical.

Smooth payoffs. With no kink, a payoff such as S^2 doesn't care where its nodes fall. A tree's average of a smooth function has an error shrinking steadily like $1/n$, with no jumps.

Exercise 22.4

The idea. Two equations in the unknown c.

Step 1. If $V_n = V + c/n$ and $V_{2n} = V + c/(2n)$, then

$$2V_{2n} - V_n = 2V + \frac{c}{n} - V - \frac{c}{n} = V.$$

Step 2. $2 \times 9.0292 - 9.0330 = 9.0254$, against the formula's 9.0248. The raw errors were 0.0082 and 0.0044; extrapolation cuts it to 0.0006.

Exercise 22.5

The idea. Add something with mean zero, and pick the multiple that cancels most noise.

Unbiased. $E[\hat{V}] = E[Y] - b(E[X] - S_0) = E[Y]$, since $E[X] = S_0$.

Its variance. For one path, $\mathrm{Var}(Y - bX) = \mathrm{Var}(Y) - 2b\,\mathrm{Cov}(Y, X) + b^2\mathrm{Var}(X)$.

The best b. This is a quadratic in b. Setting its derivative to zero gives $b^* = \mathrm{Cov}(Y, X)/\mathrm{Var}(X)$.

The minimum. Substituting:

$$\mathrm{Var}(Y) - \frac{\mathrm{Cov}(Y, X)^2}{\mathrm{Var}(X)} = \mathrm{Var}(Y)(1 - \rho^2).$$

So correlation of 0.9 removes 81% of the variance: the same accuracy with a fifth of the paths.

Exercise 22.6

The idea. A digital is a scaled coin toss, whose variance is $p(1-p)$.

One payoff. $s = e^{-rT}\sqrt{p(1-p)} = 0.97045 \times \sqrt{0.4231 \times 0.5769} = 0.4794$.

Paths for ± 0.001**.** $(1.96s/0.001)^2 = 883{,}076$.

With $p = 0.01$**.** $s = 0.0966$, so ± 0.001 takes only 35,817 paths.

But that's misleading. The rarer digital is worth only 0.0097, so ± 0.001 is a 10% error. For 10% accuracy on the first digital (worth 0.4106), you'd need just 524 paths. Rare events are hard in relative terms: few paths ever see them.

Exercise 22.7

Plain, antithetic and control-variate estimators, at three strikes.

```
import numpy as np

rng = np.random.default_rng(227)
M = 100_000
Z = rng.standard_normal(M)
grow = lambda z: S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * z)


def estimators(k):
    pay = lambda z: disc * np.maximum(grow(z) - k, 0)
    plain = pay(Z)
    # The antithetic estimator uses two payoffs per draw: compare at equal numbers of pay-
offs.
    anti = 0.5 * (pay(Z[: M // 2]) + pay(-Z[: M // 2]))
    x = disc * grow(Z)                                  # known mean: S0
    b = np.cov(plain, x)[0, 1] / x.var(ddof=1)
    ctrl = plain - b * (x - S0)
    se_p, se_a, se_c = plain.std(ddof=1) / math.sqrt(M), anti.std(ddof=1) / math.sqrt(M //
2), ctrl.std(ddof=1) / math.sqrt(M)
    return plain.mean(), se_p, anti.mean(), se_a, ctrl.mean(), se_c, np.corrcoef(plain, x)
[0, 1]


print(" strike   plain (se)         antithetic (se)    control (se)       corr    worth
(anti, ctrl)")
results = {}
for k in (85.0, 105.0, 125.0):
    p_, sp, a_, sa, c_, sc, rho = estimators(k)
    results[k] = (sp / sa) ** 2, (sp / sc) ** 2
    print(f"{k:7.0f}  {p_:7.4f} ({sp:.4f})   {a_:7.4f} ({sa:.4f})   {c_:7.4f} ({sc:.4f})
{rho:.3f}   {(sp / sa) ** 2:5.1f} x, {(sp / sc) ** 2:5.1f} x")
```

What it prints.

```
 strike   plain (se)         antithetic (se)    control (se)       corr    worth (anti, ctrl)
     85  19.1106 (0.0572)   19.1186 (0.0253)   19.1172 (0.0113)   0.980     5.1 x,  25.6 x
    105   7.1170 (0.0396)    7.1155 (0.0325)    7.1211 (0.0192)   0.875     1.5 x,   4.3 x
    125   1.9415 (0.0213)    1.9418 (0.0204)    1.9432 (0.0159)   0.666     1.1 x,   1.8 x
```

Reading it. At $K = 85$, the call behaves almost like the share (correlation 0.98), and the control variate is worth 26 times the plain paths. At $K = 125$, most paths pay nothing, the correlation is lower, and it's worth only 1.8 times. The factor is $1/(1-\rho^2)$. Antithetic helps similarly: most at the low strike, where the payoff is closest to a straight line in Z, and hardly at all at the high one.

Exercise 22.8

The American put: plain, averaged, extrapolated, and the exercise boundary.

```
def american(n, s0=S0):
    dt = T / n
    u = math.exp(sigma * math.sqrt(dt)); d = 1 / u; p_ = (math.exp(r * dt) - d) / (u - d);
df = math.exp(-r * dt)
    v = np.maximum(K - s0 * u ** (2 * np.arange(n + 1) - n), 0)
    for i in range(n - 1, -1, -1):
        v = df * (p_ * v[1:] + (1 - p_) * v[:-1])
        v = np.maximum(v, K - s0 * u ** (2 * np.arange(i + 1) - i))
    return float(v[0])


avg = lambda n, s0=S0: 0.5 * (american(n, s0) + american(n + 1, s0))
bench = avg(6400)
print(f"benchmark (n = 6400): {bench:.5f}")
print("      n   plain err   averaged err   Richardson err")
errs = {}
for n in (50, 100, 200, 400, 800):
    errs[n] = (avg(n) - bench, 2 * avg(2 * n) - avg(n) - bench)
    print(f"{n:7d}   {american(n) - bench:+9.5f}   {errs[n][0]:+12.5f}   {errs[n]
[1]:+14.5f}")

# Below the boundary the option is worth exactly its exercise value.
lo, hi = 50.0, 100.0
for _ in range(30):
    mid = 0.5 * (lo + hi)
    if american(800, mid) - (K - mid) > 1e-6:
        hi = mid
    else:
        lo = mid
print(f"exercise now if S is below about {hi:.2f}")
```

What it prints.

```
benchmark (n = 6400): 9.47747
      n   plain err   averaged err   Richardson err
     50    -0.00143       +0.01529         +0.00421
    100    +0.01003       +0.00975         +0.00038
    200    +0.00501       +0.00506         -0.00234
    400    +0.00420       +0.00136         -0.00021
    800    +0.00203       +0.00058         +0.00018
exercise now if S is below about 81.17
```

Reading it. Neighbour averaging tidies up the plain tree's jumps. Richardson usually does better still, but not reliably, since the averaged sequence isn't perfectly steady. By 800 steps, it's within about 0.0002 of the benchmark. The boundary: with one year left, exercise immediately if the share is below about 81.2. Below that level, the interest on the strike is worth more than the option's remaining flexibility.

Self-test

1. 100 times.

2. Because the error jumps with the strike's position among the nodes, so one size may be lucky. Use the worst error over a range of sizes.

3. It compares the discounted average of the children with the value of exercising, and keeps the larger.

SOLUTIONS TO LESSON 23

The Trinomial Tree

Solutions to Exercises 23.1 to 23.8 and the Lesson 23 self-test.

Exercise 23.1

The idea. The variance equation gives $p_u + p_d$, the mean equation gives $p_u - p_d$. Add and subtract.

Step 1. $(p_u + p_d)\Delta x^2 = \sigma^2 \Delta t$, with $\Delta x^2 = \lambda^2 \sigma^2 \Delta t$, gives $p_u + p_d = 1/\lambda^2$.

Step 2. The probabilities sum to 1, so $p_m = 1 - 1/\lambda^2$.

Step 3. $(p_u - p_d)\Delta x = \nu \Delta t$ gives $p_u - p_d = \nu \Delta t / \Delta x$.

Step 4. Adding and subtracting:

$$p_u = \frac{1}{2}\left(\frac{1}{\lambda^2} + \frac{\nu \Delta t}{\Delta x}\right), \qquad p_d = \frac{1}{2}\left(\frac{1}{\lambda^2} - \frac{\nu \Delta t}{\Delta x}\right).$$

In numbers. $\Delta x = \sqrt{3} \times 0.2 \times 0.1 = 0.03464$, and the tilt $\nu \Delta t / \Delta x = 0.0001 / 0.03464 = 0.00289$. So $p_u = 0.16811$, $p_m = 0.66667$, $p_d = 0.16522$.

Exercise 23.2

The idea. An odd moment depends only on $p_u - p_d$.

Step 1. On the tree. $E[\Delta X^3] = (p_u - p_d)\Delta x^3 = \dfrac{\nu \Delta t}{\Delta x}\Delta x^3 = \nu \Delta t \, \Delta x^2 = \lambda^2 \nu \sigma^2 \Delta t^2$.

Step 2. For the normal. $m^3 + 3ms^2 = \nu^3 \Delta t^3 + 3\nu\sigma^2 \Delta t^2$. To leading order, the second term: $3\nu\sigma^2 \Delta t^2$.

Step 3. They agree when $\lambda^2 = 3$. So $\lambda = \sqrt{3}$ matches the third moment as well as the fourth. With $n = 100$, both are $3 \times 0.01 \times 0.04 \times 0.0001 = 1.2 \times 10^{-7}$.

Exercise 23.3

The idea. An even moment depends only on $p_u + p_d$.

The tree. $E[\Delta X^4] = (p_u + p_d)\Delta x^4 = \dfrac{1}{\lambda^2}\lambda^4\sigma^4\Delta t^2 = \lambda^2\sigma^4\Delta t^2$.

The normal. With variance $s^2 = \sigma^2\Delta t$, and mean negligible at this order, $E[\Delta X^4] = 3s^4 = 3\sigma^4\Delta t^2$.

They agree at $\lambda = \sqrt{3}$.

The binomial. With $\lambda = 1$, a step is $\pm\sigma\sqrt{\Delta t}$, so ΔX^4 is always $\sigma^4\Delta t^2$: a coefficient of 1, not 3. A two-point step has no tails at all. (Across many steps, the sum still becomes normal, by the central limit theorem, so the tree converges all the same.)

Exercise 23.4

The idea. Write $p_d \geq 0$ as an inequality in Δt.

Step 1. $\dfrac{1}{\lambda^2} \geq \dfrac{\nu\Delta t}{\Delta x} = \dfrac{\nu\sqrt{\Delta t}}{\lambda\sigma}$. Rearranging: $\sqrt{\Delta t} \leq \dfrac{\sigma}{\lambda\nu}$, so

$$\Delta t \leq \frac{\sigma^2}{\lambda^2\nu^2}.$$

With $\lambda = \sqrt{3}$**.** For the course's numbers, $\nu = 0.01$, and the limit is $0.04/(3 \times 0.0001) = 133.3$ years: no limit at all in practice. With $r = 25\%$, $\nu = 0.23$, and $\Delta t \leq 0.252$ years, which matters for very coarse trees.

If a probability is negative. The step is no longer an average of its children. A value can then fall outside the range of its children, breaking the no-arbitrage logic. Errors can grow instead of shrink (Lesson 29).

Exercise 23.5

The idea. One step, and only one branch pays.

Step 1. The tree. $\Delta x = \sqrt{3} \times 0.2 = 0.34641$. Tilt $= 0.01/\ 0.34641$, so $p_u = 0.1811$, $p_m = 0.6667$, $p_d = 0.1522$.

Step 2. The share prices. Up: 141.40. Middle: 100. Down: 70.72.

Step 3. The payoff. Only the top branch pays: $141.40 - 105 = 36.40$.

Step 4. The price. $0.9704 \times 0.1811 \times 36.40 = 6.40$, against 7.13.

Why so poor. A single step has only three outcomes, and they can't describe the payoff's shape: the kink falls in a wide gap between 100 and 141.

The check. Average discounted share: $0.9704 \times (0.1811 \times 141.40 + 0.6667 \times 100 + 0.1522 \times 70.72) = 99.99$. It's close to 100, but not exact, as we matched log moments, not this one. On a fine tree, the gap vanishes.

Exercise 23.6

The idea. Each column of a trinomial tree grows by 2; of a binomial, by 1.

Trinomial. After i steps there are $2i + 1$ nodes. The total is $\sum_{i=0}^{n}(2i + 1) = (n + 1)^2$.

Binomial. After i steps there are $i + 1$ nodes, totalling $(n + 1)(n + 2)/2 \approx n^2/2$.

Equal work. $m^2/2 = n^2$ gives $m = \sqrt{2}\,n$ binomial steps for n trinomial steps. (If you count the operations, three multiplications per trinomial node against two, it's closer to $\sqrt{3}\,n$.)

Memory. You only need the current column: $2n + 1$ numbers for a trinomial tree, $n + 1$ for a binomial one.

Exercise 23.7

The trinomial pricer, and put-call parity on the tree.

```
import numpy as np


def tree(n, kind="call", lam_=lam, american=False):
    dt = T / n
    d_x = lam_ * sigma * math.sqrt(dt)
    tilt = nu * dt / d_x
    p_u, p_m, p_d = 0.5 * (1 / lam_**2 + tilt), 1 - 1 / lam_**2, 0.5 * (1 / lam_**2 - tilt)
    df = math.exp(-r * dt)
    pay_off = (lambda s: np.maximum(s - K, 0)) if kind == "call" else (lambda s:
np.maximum(K - s, 0))
    v = pay_off(S0 * np.exp(np.arange(-n, n + 1) * d_x))
    for i in range(n - 1, -1, -1):
        v = df * (p_u * v[2:] + p_m * v[1:-1] + p_d * v[:-2])
        if american:
            v = np.maximum(v, pay_off(S0 * np.exp(np.arange(-i, i + 1) * d_x)))
    return float(v[0])


target = S0 - K * math.exp(-r * T)
print("      n       call        put    C - P - (S0 - K e^-rT)")
gaps = {}
for n in (10, 50, 100, 500):
    c, p = tree(n), tree(n, "put")
    gaps[n] = c - p - target
    print(f"{n:7d}   {c:8.4f}   {p:8.4f}   {gaps[n]:+.2e}")
```

What it prints.

```
     n       call        put    C - P - (S0 - K e^-rT)
    10     7.2233     9.1205   -5.01e-04
    50     7.0887     8.9856   -1.00e-04
   100     7.1369     9.0337   -5.00e-05
   500     7.1271     9.0239   -1.00e-05
```

Reading it. Parity isn't exact. On a tree, parity would hold exactly if the discounted share were an exact martingale. We matched log moments instead, and $p_u e^{\Delta x} + p_m + p_d e^{-\Delta x}$ differs slightly from $e^{r\Delta t}$. The gap shrinks like $1/n$, and it's about 1.0e-05 at $n = 500$. Choosing the probabilities to match the share's own mean would make parity exact.

Exercise 23.8

Different stretch factors, then the American put.

```
bs = o["bs"]["call"]
print("  lambda    worst error, n = 200 to 209")
worst = {}
for name, l_ in (("1.2", 1.2), ("sqrt 2", math.sqrt(2)), ("sqrt 3", lam), ("2", 2.0)):
    worst[name] = max(abs(tree(n, lam_=l_) - bs) for n in range(200, 210))
    print(f"{name:>8s}  {worst[name]:.5f}")
print("      n   American put")
am = {}
for n in (100, 200, 400, 800):
    am[n] = tree(n, "put", american=True)
    print(f"{n:7d}   {am[n]:.4f}")
```

What it prints.

```
  lambda    worst error, n = 200 to 209
     1.2  0.00111
   sqrt 2  0.00443
   sqrt 3  0.00984
       2  0.00127
       n   American put
     100   9.4784
     200   9.4681
     400   9.4754
     800   9.4751
```

Reading it. The worst error isn't monotone in λ. What matters most is where the strike falls among the nodes over these ten sizes. $\lambda = \sqrt{3}$ looks worst here because $n = 200$ happens to put the strike almost on a node, where the error is steady and largest. Its virtue is theoretical (moments) and practical (stable probabilities), not a smaller raw error in every test. The American put at $n = 800$ is 9.475, close to Lesson 22's 9.478.

Self-test

1. $\frac{1}{6}, \frac{2}{3}, \frac{1}{6}$.

2. $\lambda = 1$: then $p_m = 0$.

3. Because the error becomes steady (proportional to $1/n$), and a steady error can be removed by extrapolation.

SOLUTIONS TO LESSON 24

Delta Hedging in Practice

Solutions to Exercises 24.1 to 24.8 and the Lesson 24 self-test.

Exercise 24.1

The idea. Compare the hedge's gain with the option's Taylor expansion, then use the equation to remove theta.

Step 1. The option. $\delta V \approx \Theta\,\delta t + \Delta\,\delta S + \tfrac{1}{2}\Gamma(\delta S)^2$.

Step 2. The hedge. Δ shares gain $\Delta\,\delta S$. The cash, $V - \Delta S$, earns $r(V - \Delta S)\,\delta t$.

Step 3. The seller's result. The hedge's gain minus the option's change. $\Delta\,\delta S$ cancels:

$$\text{error} \approx r(V - \Delta S)\,\delta t - \Theta\,\delta t - \tfrac{1}{2}\Gamma(\delta S)^2.$$

Step 4. The equation. From Lesson 19, $rV - rS\Delta - \Theta = \tfrac{1}{2}\sigma^2 S^2\Gamma$. So

$$\text{error} \approx \tfrac{1}{2}\sigma^2 S^2\Gamma\,\delta t - \tfrac{1}{2}\Gamma(\delta S)^2 = \tfrac{1}{2}\Gamma S^2\left(\sigma^2\delta t - \left(\tfrac{\delta S}{S}\right)^2\right).$$

Exercise 24.2

The idea. Variances of independent errors add.

Step 1. One step. With $G = \tfrac{1}{2}\Gamma S^2$ roughly constant, the variance is $G^2\mathrm{Var}((\delta S/S)^2) \approx 2G^2\sigma^4\delta t^2$.

Step 2. n steps. With $\delta t = T/n$, independent errors give

$$\mathrm{Var}(\text{total}) \approx n \times 2G^2\sigma^4\frac{T^2}{n^2} = \frac{2G^2\sigma^4T^2}{n}.$$

Step 3. So the standard deviation is proportional to $1/\sqrt{n}$.

The ratio. Between $n = 252$ and $n = 1000$: $\sqrt{1000/252} = 1.99$. The simulation gave 1.99.

Exercise 24.3

The idea. The bracket is zero when the squared move equals what was priced in.

$(\delta S/S)^2 = \sigma^2 \delta t$, so the break-even move is $\sigma\sqrt{\delta t}$.

- **One week:** $0.2 \times \sqrt{1/52} = 2.77\%$, or 2.77 on a share at 100.
- **One day:** $0.2 \times \sqrt{1/252} = 1.26\%$, or 1.26.

Why not the option. Gamma multiplies the bracket, so it sets how much is won or lost, but not where the bracket changes sign. That depends only on the volatility and the time step.

Exercise 24.4

The idea. Replace $(\delta S/S)^2$ by its true expected value in the one-step error.

Step 1. One step. The real average of $(\delta S/S)^2$ is $\sigma^2_{\text{true}}\delta t$. So the expected error at time t is

$$\tfrac{1}{2}\Gamma_t S_t^2(\sigma^2 - \sigma^2_{\text{true}})\,\delta t.$$

Step 2. Present value. Discount by e^{-rt} and add up over time:

$$\text{PV} = \tfrac{1}{2}(\sigma^2 - \sigma^2_{\text{true}}) \int_0^T e^{-rt} E[S_t^2 \Gamma_t]\, dt.$$

The direction. $\Gamma > 0$, so the integral is positive. With $\sigma > \sigma_{\text{true}}$, you overcharged, and you profit. With $\sigma < \sigma_{\text{true}}$, you undercharged, and you lose. The profit accumulates only when the share is near the strike, where gamma is large.

Exercise 24.5

The idea. The one-step error depends on the size of the move, and hardly on its average.

The drift adds $\mu\,\delta t$ to $\delta S/S$, so its square gains a term $2(\mu\,\delta t)(\sigma\sqrt{\delta t}Z) + (\mu\,\delta t)^2$. The first has mean 0, and the second is of order δt^2. Neither matters at leading order, compared with $\sigma^2\delta t$.

Where μ does enter. Through where the share goes: $E[S_t^2\Gamma_t]$ in Exercise 24.4 depends on μ. A strong drift carries the share away from the strike, so the hedger holds less gamma, which changes the spread a little.

Why this doesn't help you pick shares. The hedge deliberately removes the exposure to direction. If you have a view on μ, you'd do better to hold the share or the option unhedged.

Exercise 24.6

The idea. A first-order Taylor expansion in σ.

$\mathcal{V} \times (\sigma - \sigma_{\text{true}}) = 39.89 \times (0.20 - 0.25) = -1.994$.

The exact difference. The call at 20% is 7.1281, and at 25% it's 9.1218, a difference of -1.994. Very close, as vega is nearly constant in this range for a near-the-money option.

Exercise 24.7

Each path's final error against its realised volatility.

```
import numpy as np

Nv = np.vectorize(Nf)


def delta(s, tau, sig=sigma):
    return Nv((np.log(s / K) + (r + 0.5 * sig**2) * tau) / (sig * np.sqrt(tau)))


rng = np.random.default_rng(247)
n, paths = 252, 20_000
dt = T / n
s = np.full(paths, S0)
pos = delta(s, T)
cash = bs(sigma) - pos * s
sum_sq = np.zeros(paths)
for i in range(1, n + 1):
    ret = (mu - 0.5 * sigma**2) * dt + sigma * math.sqrt(dt) * rng.standard_normal(paths)
    s = s * np.exp(ret)
    sum_sq += ret**2
    cash *= math.exp(r * dt)
    if i < n:
        new = delta(s, T - i * dt)
        cash -= (new - pos) * s
        pos = new
err = pos * s + cash - np.maximum(s - K, 0)
real_vol = np.sqrt(sum_sq / T)

print("  realised vol        paths   mean error")
edges = [0.0, 0.19, 0.195, 0.20, 0.205, 0.21, 1.0]
for a, b in zip(edges[:-1], edges[1:]):
    m = (real_vol >= a) & (real_vol < b)
    print(f"  {a:5.3f} to {b:5.3f}   {m.sum():6d}   {err[m].mean():+10.3f}")
corr = np.corrcoef(real_vol, err)[0, 1]
print(f"correlation, all paths: {corr:.3f}")
near, far = np.abs(s - K) < 10, np.abs(s - K) > 40
c_near = np.corrcoef(real_vol[near], err[near])[0, 1]
c_far = np.corrcoef(real_vol[far], err[far])[0, 1]
print(f"finishing within 10 of the strike: {c_near:.3f}; more than 40 away: {c_far:.3f}")
```

What it prints.

```
  realised vol        paths   mean error
  0.000 to 0.190     2752       +0.586
  0.190 to 0.195     3199       +0.297
  0.195 to 0.200     4328       +0.094
  0.200 to 0.205     4213       -0.115
  0.205 to 0.210     3065       -0.309
  0.210 to 1.000     2443       -0.623
correlation, all paths: -0.806
finishing within 10 of the strike: -0.822; more than 40 away: -0.763
```

Reading it. Paths with realised volatility below 20% make money, and those above lose it, with a strong correlation of -0.81. This is the one-step formula added up: the error is the sum of $\sigma^2 \delta t - (\delta S/S)^2$, weighted by gamma. Among paths finishing near the strike (-0.82) the link is a bit tighter than among those finishing far away (-0.76), since near-the-money paths carry more gamma late on, when it matters. But it isn't a perfect link. **When** volatility was realised matters as well as how much, since the weight is gamma at that moment.

Exercise 24.8

A hedged put against a hedged call, on the same paths.

```
def hedge_err(n_reb, kind, paths_=20_000, seed=248):
    gen = np.random.default_rng(seed)
    h = T / n_reb
    x = np.full(paths_, S0)
    shift = 0.0 if kind == "call" else -1.0                # the put's delta is the call's
minus 1
    price = bs(sigma) if kind == "call" else bs(sigma) - S0 + K * math.exp(-r * T)
    q = delta(x, T) + shift
    c = price - q * x
    for i in range(1, n_reb + 1):
        x = x * np.exp((mu - 0.5 * sigma**2) * h + sigma * math.sqrt(h) *
gen.standard_normal(paths_))
        c *= math.exp(r * h)
        if i < n_reb:
            new_q = delta(x, T - i * h) + shift
            c -= (new_q - q) * x
            q = new_q
    pay_off = np.maximum(x - K, 0) if kind == "call" else np.maximum(K - x, 0)
    return q * x + c - pay_off


print("      n   sd, put   sd, call   mean, put   largest path gap")
sds = {}
for n_reb in (12, 52, 252):
    ep, ec = hedge_err(n_reb, "put"), hedge_err(n_reb, "call")
    sds[n_reb] = (ep.std(), ec.std(), np.abs(ep - ec).max())
    print(f"{n_reb:7d}   {ep.std():7.4f}   {ec.std():8.4f}   {ep.mean():+9.4f}   {sds[n_reb]
[2]:.1e}")
```

What it prints.

```
      n   sd, put   sd, call   mean, put   largest path gap
     12    2.0412     2.0412     -0.0244   2.0e-13
     52    0.9982     0.9982     -0.0039   6.7e-13
    252    0.4594     0.4594     +0.0045   2.1e-12
```

Reading it. The two standard deviations aren't just close: they're the same, path by path, to about 10^{-12}. The put hedge holds one share fewer than the call hedge, at every step. By parity, $P = C - S + Ke^{-r\tau}$: the put is a call plus a static position in the share and cash. A static position needs no rebalancing and has no hedging error, so the two errors are identical.

Self-test

1. It halves.

2. 1.26%.

3. You lose, about 2.1 at T: roughly the difference between the Black-Scholes prices at 25% and 20%, carried to expiry.

SOLUTIONS TO LESSON 25

Girsanov's Theorem

Solutions to Exercises 25.1 to 25.8 and the Lesson 25 self-test.

Exercise 25.1

The idea. W_T is normal, so its exponential has a known average.

$E[Z_T] = 1$. $W_T \sim N(0, T)$, so by the moment generating function with $u = -\theta$: $E[e^{-\theta W_T}] = e^{\frac{1}{2}\theta^2 T}$. Therefore

$$E[Z_T] = e^{-\frac{1}{2}\theta^2 T} e^{\frac{1}{2}\theta^2 T} = 1.$$

Martingale. Split $W_T = W_t + D$, with $D = W_T - W_t$ independent of $\mathcal{F}_t$ and $N(0, T - t)$.

$$E[Z_T \mid \mathcal{F}_t] = e^{-\theta W_t - \frac{1}{2}\theta^2 T} E[e^{-\theta D}] = e^{-\theta W_t - \frac{1}{2}\theta^2 T} e^{\frac{1}{2}\theta^2 (T-t)} = e^{-\theta W_t - \frac{1}{2}\theta^2 t} = Z_t.$$

Exercise 25.2

The idea. $Z_t = f(t, W_t)$ with $f(t, w) = e^{-\theta w - \frac{1}{2}\theta^2 t}$.

Step 1. The derivatives. $f_t = -\frac{1}{2}\theta^2 f$, $f_w = -\theta f$, $f_{ww} = \theta^2 f$.

Step 2. Itô's lemma.

$$dZ = \left(-\tfrac{1}{2}\theta^2 Z + \tfrac{1}{2}\theta^2 Z \right) dt - \theta Z \, dW = -\theta Z \, dW.$$

Why that means a martingale. An Itô integral $\int H \, dW$ has conditional mean zero over any future interval (Lesson 15), given integrability, which holds here. With no dt term, nothing pushes the average.

The values. At $W_T = 1$: $e^{-0.25 - 0.03125} = 0.7548$. At $W_T = -1$: $e^{0.25 - 0.03125} = 1.2445$.

Exercise 25.3

The idea. Each axiom follows from a property of Z_T.

- **Non-negative:** $Z_T > 0$ and $\mathbf{1}_A \geq 0$, so $E[Z_T \mathbf{1}_A] \geq 0$.

- **Total 1:** $\tilde{P}(\Omega) = E[Z_T] = 1$.
- **Additive:** for disjoint events, $\mathbf{1}_{A\cup B} = \mathbf{1}_A + \mathbf{1}_B$, and an average is additive. (Likewise for countable unions, by monotone convergence.)

Equivalent. If $P(A) = 0$, then $Z_T \mathbf{1}_A = 0$ with probability 1, so $\tilde{P}(A) = 0$. If $\tilde{P}(A) = 0$, then $Z_T \mathbf{1}_A$ is non-negative with zero average, so it's zero with probability 1. Since $Z_T > 0$, $\mathbf{1}_A = 0$ with probability 1, so $P(A) = 0$.

If Z_T could be 0. An event possible under P could be impossible under $\tilde{P}$. The reverse-direction argument fails, and the two would no longer be equivalent. Under $\tilde{P}$, a hedge could be missing some scenarios that can really happen.

Exercise 25.4

The idea. Combine the exponents, and the MGF does the rest.

Step 1. $Z_T e^{u(W_T+\theta T)} = \exp\big((u-\theta)W_T + u\theta T - \frac{1}{2}\theta^2 T\big)$.

Step 2. Average over $W_T \sim N(0,T)$: $E[e^{(u-\theta)W_T}] = e^{\frac{1}{2}(u-\theta)^2 T}$.

Step 3. The total exponent:

$$\tfrac{1}{2}(u^2 - 2u\theta + \theta^2)T + u\theta T - \tfrac{1}{2}\theta^2 T = \tfrac{1}{2}u^2 T.$$

So $\tilde{E}[e^{u\tilde{W}_T}] = e^{\frac{1}{2}u^2 T}$, the MGF of $N(0,T)$. An MGF determines the distribution, so $\tilde{W}_T \sim N(0,T)$ under $\tilde{P}$.

Exercise 25.5

The idea. Substitute $dW = d\tilde{W} - \theta\, dt$.

Step 1. $dS = \mu S\, dt + \sigma S(d\tilde{W} - \theta\, dt) = (\mu - \sigma\theta)S\, dt + \sigma S\, d\tilde{W} = rS\, dt + \sigma S\, d\tilde{W}$.

Step 2. Solve, as in Lesson 17: $S_T = S_0 e^{(r-\frac{1}{2}\sigma^2)T + \sigma\tilde{W}_T}$.

Step 3. Under $\tilde{P}$. $\tilde{W}_T \sim N(0,T)$, so

$$\tilde{P}(S_T > K) = N\Big(\frac{\ln(S_0/K) + (r - \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}\Big) = N(-0.1940) = 0.4231.$$

Step 4. Under P. $S_T = S_0 e^{(\mu-\frac{1}{2}\sigma^2)T + \sigma W_T}$, so the same formula with μ gives $N(0.0560) = 0.5223$.

The real-world chance is higher by 0.0992: the same event, weighted differently.

Exercise 25.6

The idea. Z_T^2 is again an exponential of a normal.

Step 1. $Z_T^2 = e^{-2\theta W_T - \theta^2 T}$. With $E[e^{-2\theta W_T}] = e^{2\theta^2 T}$, we get $E[Z_T^2] = e^{\theta^2 T}$.

Step 2. The numbers.

- $\theta = 0.25$: $E[Z_T^2] = 1.0645$, and the effective share of paths is 0.939.
- $\theta = 3$: $E[Z_T^2] = 8103$, and the effective share is 1.2e-04. From a million paths, about 123 effective ones.

Why it's dangerous. With a large θ, nearly all the weight sits on a few paths far out in the tail. The estimate rests on those few, and they may not even have been sampled. Worse, a run that misses them looks calm, and its own error bar can be badly wrong.

Exercise 25.7

The call and the put, with and without the weights.

```
import numpy as np

rng = np.random.default_rng(257)
M = 1_000_000
Z = rng.standard_normal(M)
disc = math.exp(-r * T)
pays = {"call": lambda s: disc * np.maximum(s - K, 0), "put": lambda s: disc * np.maximum(K
- s, 0)}
print(f"formula: call {o['bs']['call']:.4f}, put {o['bs']['put']:.4f}")
print("                                    call (se)            put (se)")
S_rn = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * Z)
a, b = pays["call"](S_rn), pays["put"](S_rn)
print(f"directly under the new probability  {a.mean():7.4f} ({a.std() / 1000:.4f})
{b.mean():7.4f} ({b.std() / 1000:.4f})")
se = {}
for mu in (0.08, 0.50):
    theta = (mu - r) / sigma
    W = math.sqrt(T) * Z                                    # W is Brownian under P
    S_T = S0 * np.exp((mu - 0.5 * sigma**2) * T + sigma * W)
    weight = np.exp(-theta * W - 0.5 * theta**2 * T)
    a, b = pays["call"](S_T) * weight, pays["put"](S_T) * weight
    se[mu] = (a.std() / 1000, b.std() / 1000)
    print(f"under P, mu = {mu:3.0%}, weighted           {a.mean():7.4f} ({se[mu][0]:.4f})
{b.mean():7.4f} ({se[mu][1]:.4f})")
```

What it prints.

```
formula: call 7.1281, put 9.0248
                                      call (se)            put (se)
directly under the new probability   7.1551 (0.0126)    9.0174 (0.0111)
under P, mu =  8%, weighted             7.1474 (0.0094)    9.0160 (0.0149)
under P, mu = 50%, weighted             7.1296 (0.0088)    8.6541 (0.3488)
```

Reading it. This one rewards a careful look, and it isn't what the last exercise might lead you to expect.

The call's error shrinks as μ rises: 0.0094 at 8%, 0.0088 at 50%. Paths drifting up land more often where the call pays. There the weights are small, and that keeps the product under control. This is **importance sampling**, used well: sample more of what matters, and correct with weights.

The put's error explodes: 0.0149 at 8%, 0.3488 at 50%. Now the paths rarely land where the put pays, and when they do, the weight is enormous. The estimate is far from 9.02, and its reported error is itself unreliable, as it depends on the few paths that dominate.

The lesson. A change of probability isn't good or bad in itself. Its effect depends on whether it pushes samples towards the payoff, or away from it.

Exercise 25.8

Quadratic variation, with and without the weights.

```
rng = np.random.default_rng(258)
paths, n = 2000, 2000
dW = rng.normal(0.0, math.sqrt(T / n), (paths, n))
Z_T = np.exp(-th * dW.sum(axis=1) - 0.5 * th**2 * T)
w = Z_T / Z_T.sum()
qv = (dW**2).sum(axis=1)


def weighted_quantile(x, weights, q):
    order = np.argsort(x)
    return float(x[order][np.searchsorted(weights[order].cumsum(), q)])


print("            plain     weighted")
print(f"mean QV    {qv.mean():8.4f}  {(w * qv).sum():8.4f}")
for q in (0.05, 0.5, 0.95):
    print(f"{q:4.0%} QV   {np.quantile(qv, q):8.4f}  {weighted_quantile(qv, w, q):8.4f}")

half = dW[:, : n // 2].sum(axis=1)
Wt_half, Wt_end = half + th * 0.5, dW.sum(axis=1) + th * T
print(f"weighted mean of (W tilde)^2 - t: t = 0.5: {(w * (Wt_half**2 - 0.5)).sum():+.4f}, t =
1: {(w * (Wt_end**2 - 1)).sum():+.4f}")
```

What it prints.

```
            plain     weighted
mean QV      0.9993    0.9997
  5% QV     0.9488    0.9492
 50% QV     0.9988    0.9990
 95% QV     1.0540    1.0546
weighted mean of (W tilde)^2 - t: t = 0.5: -0.0205, t = 1: -0.0134
```

Reading it. The mean and all three quantiles of the realised $\sum(\Delta W)^2$ barely change with the weights, as they couldn't: quadratic variation is a property of each path, and it's nearly the same number on all of them. The martingale $\tilde{W}_t^2 - t$ has a weighted mean within sampling error of zero, as it should for a Brownian motion under $\tilde{P}$. (With only 2000 paths the noise in those two means is about 0.03.)

Self-test

1. -0.25.

2. $\tilde{W}_t = W_t + \theta t$.

3. Because quadratic variation is a property of each path, holding with probability 1. A change of probability reweights the paths, and leaves them unchanged. An equivalent probability has the same probability-1 events.

SOLUTIONS TO LESSON 26

Risk-Neutral Pricing

Solutions to Exercises 26.1 to 26.8 and the Lesson 26 self-test.

Exercise 26.1

The idea. The product rule, with a discount factor that has no randomness.

Step 1. $D_t = e^{-rt}$ has $dD = -rD\,dt$, with no dW part.

Step 2. So $dD\,dS = -rD\,dt\,(\mu S\,dt + \sigma S\,dW) = 0$ by the multiplication table: $dt\,dt = dt\,dW = 0$. **That's why there's no cross-term.**

Step 3.

$$d(DS) = D\,dS + S\,dD = D(\mu S\,dt + \sigma S\,dW) - rDS\,dt = (\mu - r)DS\,dt + \sigma DS\,dW.$$

Step 4. Factor out σDS: $\sigma DS\left(\frac{\mu - r}{\sigma}\,dt + dW\right) = \sigma DS(\theta\,dt + dW)$.

Exercise 26.2

The idea. θ measures the reward per unit of risk.

The value. $\theta = (0.08 - 0.03)/0.2 = 0.25$.

$\theta = 0$. It means $\mu = r$: no reward for risk. Then $Z_T = 1$, so $\tilde{P} = P$. The real world is already risk-neutral.

The average price. Under $\tilde{P}$, the drift is r: $\tilde{E}[S_T] = S_0 e^{rT} = 103.05$. Under P, the drift is μ: $E[S_T] = S_0 e^{\mu T} = 108.33$.

Exercise 26.3

The idea. Discounting removes the cash, leaving only the discounted share.

Step 1. From Exercise 19.1: $d(DX) = \Delta\,d(DS)$.

Step 2. So $d(DX) = \Delta\sigma DS\,d\tilde{W}$, with no dt term. For any reasonable Δ, this is a $\tilde{P}$-martingale.

No sure profit. Suppose $X_0 = 0$, $X_T \geq 0$ with probability 1, and $P(X_T > 0) > 0$. Since the two probabilities are equivalent, the same holds under $\tilde{P}$. So $\tilde{E}[D_T X_T] > 0$. But the martingale property says $\tilde{E}[D_T X_T] = X_0 = 0$. Contradiction.

Note what's used. It doesn't matter how clever Δ is: the argument applies to all of them, as long as Δ uses only the past.

Exercise 26.4

The idea. The discounted share is a martingale, so its average is S_0.

$$V_0 = \tilde{E}[e^{-rT}(S_T - F)] = \tilde{E}[e^{-rT} S_T] - e^{-rT} F = S_0 - e^{-rT} F.$$

Zero when $F = S_0 e^{rT} = 103.0455$.

What wasn't needed. Neither σ nor μ. The forward can be hedged statically: buy the share, borrow its price. It doesn't depend on any model of how the share moves.

Exercise 26.5

The idea. Itô on $\tilde{W}^2$, as in Lesson 15.

Step 1. $d(\tilde{W}_t^2) = 2\tilde{W}_t \, d\tilde{W}_t + dt$. Integrating from 0 to T: $\tilde{W}_T^2 - T = \int_0^T 2\tilde{W}_t \, d\tilde{W}_t$.

Step 2. So $\Gamma_t = 2\tilde{W}_t$.

The hedge. With $r = 0$, $D = 1$, and $\Delta_t = \Gamma_t / (\sigma S_t) = \dfrac{2\tilde{W}_t}{\sigma S_t}$ shares. When $\tilde{W}_t$ is large in size, the claim's value is sensitive to further moves, so more shares are needed.

Exercise 26.6

The idea. Choose weights that cancel the volatility. A riskless portfolio must earn r.

Step 1. Weights. Put w_1 of your wealth in asset 1 and $w_2 = 1 - w_1$ in asset 2. Its return is $(w_1 \mu_1 + w_2 \mu_2)\, dt + (w_1 \sigma_1 + w_2 \sigma_2)\, dW$.

Step 2. Cancel dW**.** $w_1 \sigma_1 + (1 - w_1)\sigma_2 = 0$, so $w_1 = \sigma_2 / (\sigma_2 - \sigma_1) = 0.3 / 0.1 = 3$, and $w_2 =$ -2.

Step 3. Its return. $3 \times 8\% - 2 \times 14\% =$ -0.04, or −4% a year, with no risk at all. Compare with cash at 3%.

The arbitrage. Short this portfolio (so hold -3 of asset 1 and $+2$ of asset 2 per unit), and invest the proceeds in cash. You earn $3\% - (-4\%) = 7\%$ a year, with no risk and no money down.

Which μ_2 removes it? The two market prices of risk must be equal: $(\mu_2 - r)/\sigma_2 = (\mu_1 - r)/\sigma_1 = 0.25$. So $\mu_2 = 3\% + 0.25 \times 30\% = 0.105$, or 10.5%.

Exercise 26.7

Two payoffs, each simulated both ways.

```
import numpy as np

theta = (mu - r) / sigma
dm = (math.log(S0 / K) + (r - 0.5 * sigma**2) * T) / sigma
dp = dm + sigma
disc = math.exp(-r * T)
exact = {"call": S0 * Nf(dp) - K * disc * Nf(dm), "digital": disc * Nf(dm)}

rng = np.random.default_rng(267)
M = 1_000_000
Z = rng.standard_normal(M)
S_rn = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * Z)
S_re = S0 * np.exp((mu - 0.5 * sigma**2) * T + sigma * Z)
wt = np.exp(-theta * Z - 0.5 * theta**2 * T)
pays = {"call": lambda s: disc * np.maximum(s - K, 0), "digital": lambda s: disc * (s > K)}
print("          exact     risk-neutral (se)     real with weights (se)   se ratio")
ratios = {}
for name, f in pays.items():
    a, b = f(S_rn), f(S_re) * wt
    ratios[name] = (b.std() / a.std())
    print(f"{name:8s}  {exact[name]:7.4f}  {a.mean():7.4f} ({a.std() / 1000:.5f})
{b.mean():7.4f} ({b.std() / 1000:.5f})      {ratios[name]:.2f}")
```

What it prints.

```
          exact     risk-neutral (se)     real with weights (se)   se ratio
call       7.1281   7.1100 (0.01253)     7.1112 (0.00937)      0.75
digital    0.4106   0.4095 (0.00048)     0.4100 (0.00040)      0.84
```

Reading it. Here, weights don't hurt: the weighted standard errors are lower, by factors of 0.75 for the call and 0.84 for the digital. Both payoffs live on high share prices. Simulating at $\mu = 8\%$ lands more paths there, and the weights there are below 1. The call gains more, since it has more weight in the far right tail. This is importance sampling, as in Exercise 25.7.

Exercise 26.8

Rebuilding the call's discounted payoff from the martingale's integral.

```
Nv = np.vectorize(Nf)


def bs_call(s, tau):
    d1 = (np.log(s / K) + (r + 0.5 * sigma**2) * tau) / (sigma * np.sqrt(tau))
    return s * Nv(d1) - K * np.exp(-r * tau) * Nv(d1 - sigma * np.sqrt(tau))


rng = np.random.default_rng(268)
paths = 200
fine = 2000
dW = rng.normal(0.0, math.sqrt(T / fine), (paths, fine))
print("  steps   rms gap")
rms = {}
for n in (500, 1000, 2000):
    h = T / n
    inc = dW.reshape(paths, n, fine // n).sum(axis=2)         # the same paths at n coarser
steps
    s = np.full(paths, S0)
    M_ = np.full(paths, float(bs_call(S0, T)))
    for i in range(n):
        t = i * h
        d1 = (np.log(s / K) + (r + 0.5 * sigma**2) * (T - t)) / (sigma * math.sqrt(T - t))
        M_ += sigma * math.exp(-r * t) * s * Nv(d1) * inc[:, i]
        s = s * np.exp((r - 0.5 * sigma**2) * h + sigma * inc[:, i])
    gap = M_ - math.exp(-r * T) * np.maximum(s - K, 0)
    rms[n] = float(np.sqrt((gap**2).mean()))
    print(f"{n:7d}   {rms[n]:.4f}")
```

What it prints.

```
  steps   rms gap
    500   0.4147
   1000   0.2892
   2000   0.2061
```

Reading it. The root-mean-square gap falls by about $\sqrt{2}$ each time the steps double, so like $1/\sqrt{n}$. It's the same rate as the hedging error of Lesson 24, and for the same reason: $M_0 + \sum \Gamma \, \Delta\tilde{W}$ is the discounted wealth of a discretely rebalanced delta hedge. Over 500 to 2000 steps, the error falls by 2.01, close to $\sqrt{4} = 2$.

Self-test

1. $\theta = 0.25$: the share's excess return per unit of volatility, its Sharpe ratio.

2. Zero.

3. The existence of a hedge, and its form. The pricing formula says what a replicable payoff must be worth. Representation says that every payoff is replicable, and how.

SOLUTIONS TO LESSON 27

Change of Numéraire

Solutions to Exercises 27.1 to 27.8 and the Lesson 27 self-test.

Exercise 27.1

The idea. Two different arguments, each short.

By the martingale property. Under $\tilde{P}$, $e^{-rt}S_t$ is a martingale starting at S_0. So $\tilde{E}[e^{-rT}S_T] = S_0$, and dividing by S_0 gives 1.

By the normal. $e^{-rT}S_T/S_0 = e^{-\frac{1}{2}\sigma^2 T + \sigma\tilde{W}_T}$. With $\tilde{W}_T \sim N(0,T)$, the MGF gives $E[e^{\sigma\tilde{W}_T}] = e^{\frac{1}{2}\sigma^2 T}$. The product is 1.

Exercise 27.2

The idea. Compute the MGF of $\ln S_T$ under P^S, with the weight.

Step 1. Let $Y = \ln(S_T/S_0) = (r - \frac{1}{2}\sigma^2)T + \sigma\tilde{W}_T$. The weight is $Z_T^S = e^{\sigma\tilde{W}_T - \frac{1}{2}\sigma^2 T}$.

Step 2.

$$E^S[e^{uY}] = \tilde{E}[Z_T^S e^{uY}] = e^{u(r-\frac{1}{2}\sigma^2)T - \frac{1}{2}\sigma^2 T}\tilde{E}[e^{(u+1)\sigma\tilde{W}_T}].$$

Step 3. The MGF gives $\tilde{E}[e^{(u+1)\sigma\tilde{W}_T}] = e^{\frac{1}{2}(u+1)^2\sigma^2 T}$. Collecting the exponents:

$$u(r - \tfrac{1}{2}\sigma^2)T - \tfrac{1}{2}\sigma^2 T + \tfrac{1}{2}(u^2 + 2u + 1)\sigma^2 T = u(r + \tfrac{1}{2}\sigma^2)T + \tfrac{1}{2}u^2\sigma^2 T.$$

This is the MGF of $N\big((r + \frac{1}{2}\sigma^2)T,\ \sigma^2 T\big)$. So $\ln S_T$ is normal with mean $\ln S_0 + (r + \frac{1}{2}\sigma^2)T$ and variance $\sigma^2 T$.

The median of S_T is $S_0 e^{(r+\frac{1}{2}\sigma^2)T} = 100e^{0.05} = 105.13$.

Exercise 27.3

The idea. Write $Y = f(t, S)$, and let Itô's lemma show the drift is zero.

Step 1. $f(t,S) = e^{rt}/S$: $f_t = rf$, $f_S = -f/S$, $f_{SS} = 2f/S^2$.

Step 2. Itô's lemma, with $a = (r + \sigma^2)S$ and $b = \sigma S$:

$$dY = \left(rY - (r+\sigma^2)Y + \tfrac{1}{2}\sigma^2 S^2 \frac{2Y}{S^2}\right) dt - \sigma Y\, dW^S.$$

Step 3. The dt bracket is $rY - rY - \sigma^2 Y + \sigma^2 Y = 0$. So $dY = -\sigma Y\, dW^S$: a martingale under P^S.

Exercise 27.4

The idea. Under P^S the drift is different, but it's the same standardising calculation.

Step 1. Under P^S, $\ln S_T = \ln S_0 + (r + \tfrac{1}{2}\sigma^2)T + \sigma\sqrt{T}Z$ with $Z \sim N(0,1)$.

Step 2. $S_T > K \iff Z > -\dfrac{\ln(S_0/K) + (r+\frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}} = -d_+$. So $P^S(S_T > K) = N(d_+)$.

The two terms.

- The strike term $K\mathbf{1}\{S_T > K\}$ is a fixed sum in cash, so use the cash numéraire: $Ke^{-rT}\tilde{P}(S_T > K) = Ke^{-rT}N(d_-)$.
- The share term $S_T\mathbf{1}\{S_T > K\}$ is a fixed number of shares, so use the share numéraire: $S_0 E^S[\mathbf{1}\{S_T > K\}] = S_0 N(d_+)$.

Why $N(d_+) > N(d_-)$. P^S gives each path a weight proportional to S_T, so it favours exactly the paths on which the share ends high. Those are the in-the-money ones. So the same event has a higher probability.

Exercise 27.5

The idea. In units of asset 2, this is an ordinary call with strike 1 and a zero rate.

Step 1. The numéraire. Use P^2, with weight $Z = e^{-rT}S_T^2/S_0^2$. By the argument of Lesson 27,

$$V_0 = S_0^2 E^2\left[\frac{\max(S_T^1 - S_T^2, 0)}{S_T^2}\right] = S_0^2 E^2[\max(Y_T - 1, 0)], \qquad Y = S^1/S^2.$$

Step 2. Y's motion. Y is a traded asset counted in the numéraire, so it's a P^2-martingale: no drift.

Step 3. Y's volatility. With the risk-neutral forms $dS^i = rS^i\,dt + \sigma_i S^i\, d\tilde{W}^i$, the random part of $d\ln Y = d\ln S^1 - d\ln S^2$ is $\sigma_1\, d\tilde{W}^1 - \sigma_2\, d\tilde{W}^2$. Its quadratic variation per unit time is

$$\sigma_x^2 = \sigma_1^2 - 2\rho\sigma_1\sigma_2 + \sigma_2^2.$$

Change of probability doesn't affect this (Lesson 25).

Step 4. Read off. Y is lognormal with volatility σ_x, no drift, and starts at $Y_0 = S_0^1/S_0^2$. So $E^2[\max(Y_T - 1, 0)]$ is the Black-Scholes call with strike 1 and $r = 0$: $Y_0 N(d_1) - N(d_2)$. Multiplying by S_0^2 gives $V_0 = S_0^1 N(d_1) - S_0^2 N(d_2)$.

In numbers. $\sigma_x^2 = 0.04 + 0.09 - 2 \times 0.5 \times 0.06 = 0.07$, so $\sigma_x = 0.2646$. And $d_1 = 0.3262$, $d_2 = 0.0616$, $N(d_1) = 0.6278$, $N(d_2) = 0.5246$. So

$$V_0 = 100 \times 0.6278 - 95 \times 0.5246 = 12.9523.$$

Exercise 27.6

The idea. The put pays when the share ends below K, so both probabilities are of that event.

Split. $P = Ke^{-rT}\tilde{P}(S_T < K) - S_0 P^S(S_T < K)$, by the same argument as the call, with cash on the first term, and shares on the second.

The probabilities. $\tilde{P}(S_T < K) = 1 - N(d_-) = N(-d_-) = 0.5769$, and $P^S(S_T < K) = N(-d_+) = 0.4976$.

Why "below". The put pays only there. The share unit lowers the chance of finishing below K, as it favours high paths: $N(-d_+) < N(-d_-)$.

Exercise 27.7

N(d₊), estimated directly and with weights.

```
import numpy as np

M = 1_000_000
print(" strike    N(d+)     under P^S (se)        weighted from Q (se)")
ses = {}
for k in (105.0, 160.0):
    dp = (math.log(S0 / k) + (r + 0.5 * sigma**2) * T) / sigma
    rng = np.random.default_rng(277)
    Z = rng.standard_normal(M)
    direct = (S0 * np.exp((r + 0.5 * sigma**2) * T + sigma * Z) > k).astype(float)
    S_Q = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * Z)
    weighted = math.exp(-r * T) * S_Q / S0 * (S_Q > k)
    ses[k] = (direct.std() / 1000, weighted.std() / 1000)
    print(f"{k:7.0f}  {N(dp):8.5f}  {direct.mean():8.5f} ({ses[k][0]:.5f})
{weighted.mean():8.5f} ({ses[k][1]:.5f})")
```

What it prints.

```
 strike    N(d+)     under P^S (se)        weighted from Q (se)
    105   0.50241   0.50235 (0.00050)     0.50208 (0.00059)
    160   0.01786   0.01781 (0.00013)     0.01784 (0.00017)
```

Reading it. Direct simulation has the lower standard error here, by a factor of 1.19 at $K = 105$ and 1.30 at $K = 160$. The direct estimator is a pure coin toss, with variance $p(1-p)$. The weighted one multiplies the coin by a weight that varies from path to path, which adds variance. The weights do some good at $K = 160$, by putting more value on the paths that matter, but not enough to beat a direct draw from the right distribution. When you can simulate directly under the right probability, do.

Exercise 27.8

Margrabe's formula against Monte Carlo.

```python
def margrabe(a0, b0, s_a, s_b, rho):
    v = math.sqrt(s_a**2 + s_b**2 - 2 * rho * s_a * s_b)
    d_1 = (math.log(a0 / b0) + 0.5 * v**2 * T) / (v * math.sqrt(T))
    return a0 * N(d_1) - b0 * N(d_1 - v * math.sqrt(T))


a0, b0, s_a, s_b = 100.0, 95.0, 0.2, 0.3
rng = np.random.default_rng(278)
z1, z_ind = rng.standard_normal(M), rng.standard_normal(M)
print("   rho   formula   simulated, r = 0     r = 10%")
vals = {}
for rho in (-0.5, 0.0, 0.5, 0.9):
    z2 = rho * z1 + math.sqrt(1 - rho**2) * z_ind
    res = []
    for rate in (0.0, 0.10):
        A = a0 * np.exp((rate - 0.5 * s_a**2) * T + s_a * z1)
        B = b0 * np.exp((rate - 0.5 * s_b**2) * T + s_b * z2)
        res.append(math.exp(-rate * T) * np.maximum(A - B, 0).mean())
    vals[rho] = margrabe(a0, b0, s_a, s_b, rho)
    print(f"{rho:6.1f}   {vals[rho]:7.4f}   {res[0]:16.4f}   {res[1]:9.4f}")
    assert abs(res[0] - vals[rho]) < 0.08 and abs(res[0] - res[1]) < 1e-9
```

What it prints.

```
   rho   formula   simulated, r = 0     r = 10%
  -0.5   19.4360           19.4522     19.4522
   0.0   16.5882           16.6041     16.6041
   0.5   12.9523           12.9656     12.9656
   0.9    8.6046            8.6143      8.6143
```

Reading it. The formula matches within sampling error. The price **falls** as ρ rises. When the two assets move together, their ratio hardly moves, so there's little optionality: $\sigma_x^2 = \sigma_1^2 + \sigma_2^2 - 2\rho\sigma_1\sigma_2$ shrinks. At $\rho = -0.5$, the assets diverge, and the right to swap is worth much more.

The rate. Changing r from 0 to 10% changes nothing: on the same random numbers, the results agree exactly. Both assets get the factor e^{rT}, and the discount e^{-rT} cancels it.

Self-test

1. $r + \sigma^2$, which is 7% with the course's numbers.

2. The probability that the call finishes in the money, in share units: 0.5024. (In cash units, it's 0.4231.)

3. Both assets grow at the same risk-neutral rate, so only their ratio matters. Counted in units of asset 2, the ratio has no drift, and the discount has gone.

SOLUTIONS TO LESSON 28

The Feynman-Kac Theorem

Solutions to Exercises 28.1 to 28.8 and the Lesson 28 self-test.

Exercise 28.1

The idea. Conditional on today, a forecast of a forecast is a forecast.

Martingale. Let $Y_t = E[h(X_T) \mid \mathcal{F}_t]$. For $s < t$, the tower property gives

$$E[Y_t \mid \mathcal{F}_s] = E\big[E[h(X_T) \mid \mathcal{F}_t] \mid \mathcal{F}_s\big] = E[h(X_T) \mid \mathcal{F}_s] = Y_s.$$

The Markov property. Y_t could in principle depend on the whole path up to t. With $dX = a(X)\,dt + b(X)\,dW$, future moves depend only on the present value and fresh noise. So the conditional expectation depends on the past only through X_t, and $Y_t = g(t, X_t)$ for an ordinary function g. Without this, there'd be no function of (t, x) to differentiate.

Exercise 28.2

The idea. With $a = 0$, $b = 1$: $g_t + \frac{1}{2} g_{xx} = 0$.

$h = x^2$**.** Guess $g = x^2 + c(t)$. Then $g_t = c'$, $g_{xx} = 2$, so $c' + 1 = 0$. With $g(T, x) = x^2$, we need $c(T) = 0$, so $c = T - t$. Hence $g = x^2 + T - t$.

Check. $E[(x + \sqrt{T-t}\,Z)^2] = x^2 + (T - t)$. ✓

$h = e^x$**.** Guess $g = e^{x + c(t)}$. Then $g_t = c' g$, $g_{xx} = g$, so $c' + \frac{1}{2} = 0$. With $c(T) = 0$, $c = \frac{1}{2}(T - t)$. Hence $g = e^{x + \frac{1}{2}(T-t)}$.

Check. $E[e^{x + \sqrt{T-t}\,Z}] = e^x e^{\frac{1}{2}(T-t)}$, by the normal's MGF. ✓

Exercise 28.3

The idea. It's the discounted process that's the martingale.

Step 1. $e^{-rt} g(t, X_t) = e^{-rt} e^{-r(T-t)} E[h(X_T) \mid \mathcal{F}_t] = e^{-rT} E[h(X_T) \mid \mathcal{F}_t]$. This is a constant times a conditional expectation, and so is a martingale, by the tower property.

Step 2. Product rule and Itô.

$$d(e^{-rt}g) = e^{-rt}\,dg - re^{-rt}g\,dt = e^{-rt}\big(g_t + ag_x + \tfrac{1}{2}b^2 g_{xx} - rg\big)\,dt + e^{-rt}bg_x\,dW.$$

Step 3. A martingale has no dt term, and $e^{-rt} > 0$. So $g_t + ag_x + \frac{1}{2}b^2 g_{xx} = rg$.

Exercise 28.4

The idea. The equation is linear in S, so the solution is S times a function of time.

Undiscounted. $g_t + \mu S g_S + \frac{1}{2}\sigma^2 S^2 g_{SS} = 0$. With $g = Sc(t)$: $g_t = Sc'$, $g_S = c$, $g_{SS} = 0$. So $c' + \mu c = 0$, with $c(T) = 1$. Hence $c = e^{\mu(T-t)}$, and $g = Se^{\mu(T-t)}$. With our numbers, from 100 at $t = 0$: 108.33.

Discounted. The right side becomes rg: $c' + \mu c = rc$. So $c = e^{(\mu-r)(T-t)}$, and $g = Se^{(\mu-r)(T-t)}$, which is 105.13.

When $\mu = r$: $c = 1$, so $g = S$. The discounted average of the share is today's share price. A share is worth its own price.

Exercise 28.5

The idea. Each expectation solves its own equation. The theorem doesn't say which one is the price.

What's right. Feynman-Kac is about any drift a. Pick $a = \mu S$, and $g_\mu = E[e^{-r(T-t)}h(S_T) \mid S_t = S]$ solves $g_t + \mu S g_S + \frac{1}{2}\sigma^2 S^2 g_{SS} = rg$.

What's wrong. That isn't the Black-Scholes equation, which has r where this has μ. The hedging argument of Lesson 19 says the option's value must solve the equation with r, and so, by Feynman-Kac with $a = rS$, it's the expectation with drift r.

The extra term. Put g_μ into the Black-Scholes equation: from its own equation, $g_t + \frac{1}{2}\sigma^2 S^2 g_{SS} = rg - \mu S g_S$. So

$$g_t + rSg_S + \tfrac{1}{2}\sigma^2 S^2 g_{SS} - rg = -(\mu - r)Sg_S.$$

It fails by $(\mu - r)Sg_S$, which at $S = 100$, $t = 0.5$ is 2.5808. And in value, 9.96 isn't the option's price, 7.13.

Exercise 28.6

The idea. The equation must hold for every r, so the coefficient of r and the rest must vanish separately.

Step 1. Derivatives. With $P = e^{A(\tau)-B(\tau)r}$ and $\tau = T - t$: $P_t = -(A' - B'r)P$, $P_r = -BP$, $P_{rr} = B^2 P$.

Step 2. Substitute, and divide by P.

$$-A' + B'r - \kappa(\bar{r} - r)B + \tfrac{1}{2}s^2 B^2 = r.$$

Step 3. Terms with r: $B' + \kappa B = 1$, with $B(0) = 0$. So $B = (1 - e^{-\kappa\tau})/\kappa$.

Step 4. Terms without r: $A' = -\kappa\bar{r}B + \frac{1}{2}s^2 B^2$, with $A(0) = 0$.

Step 5. Integrate. Using $\int_0^\tau B\,du = (\tau - B)/\kappa$, and $\int_0^\tau B^2\,du = \dfrac{1}{\kappa^2}(\tau - B) - \dfrac{B^2}{2\kappa}$:

$$A = -\bar{r}(\tau - B) + \frac{s^2}{2\kappa^2}(\tau - B) - \frac{s^2 B^2}{4\kappa} = \left(\bar{r} - \frac{s^2}{2\kappa^2}\right)(B - \tau) - \frac{s^2 B^2}{4\kappa}.$$

✓ This is the formula in Lesson 28.

Exercise 28.7

By hand. With $g = x^4 + c_1 x^2 + c_2$: $g_t = c_1' x^2 + c_2'$, $g_{xx} = 12x^2 + 2c_1$. The equation $g_t + \frac{1}{2}g_{xx} = 0$ gives

$$(c_1' + 6)x^2 + (c_2' + c_1) = 0,$$

for every x. So $c_1' = -6$ and $c_2' = -c_1$, with both zero at T. Hence $c_1 = 6\tau$, and $c_2 = 3\tau^2$, with $\tau = T - t$.

The polynomial, against simulation and against the equation.

```
import numpy as np

g = lambda t, x: x**4 + 6 * (T - t) * x**2 + 3 * (T - t) ** 2      # c1 = 6 tau, c2 = 3 tau^2


def residual(t, x, h=1e-3, k=1e-4):
    gt = (g(t + k, x) - g(t - k, x)) / (2 * k)
    gxx = (g(t, x + h) - 2 * g(t, x) + g(t, x - h)) / h**2
    return gt + 0.5 * gxx


rng = np.random.default_rng(287)
Z = rng.standard_normal(2_000_000)
print("    t     x      formula    simulated    PDE residual")
gaps = []
for t, x in ((0.0, 0.0), (0.0, 1.0), (0.5, 1.0), (0.5, -2.0), (0.9, 0.5)):
    sim = ((x + math.sqrt(T - t) * Z) ** 4).mean()
    gaps.append(abs(sim - g(t, x)) / max(g(t, x), 0.1))
    print(f"{t:5.1f} {x:5.1f}   {g(t, x):10.4f}   {sim:10.4f}    {residual(t, x):+.1e}")
```

What it prints.

```
    t     x      formula    simulated    PDE residual
  0.0   0.0       3.0000       3.0016    +1.0e-06
  0.0   1.0      10.0000      10.0168    +1.0e-06
  0.5   1.0       4.7500       4.7566    +1.0e-06
  0.5  -2.0      28.7500      28.7346    +1.0e-06
  0.9   0.5       0.2425       0.2428    +1.0e-06
```

Reading it. All agree. At $x = 0$, $t = 0$, $g = 3$: the normal's fourth moment again (Lesson 4), now as a PDE solution.

Exercise 28.8

Vasicek bonds by Monte Carlo, at three step sizes, with two integrals.

```
kappa, r_bar, s, r0 = 0.5, 0.04, 0.01, 0.03


def formula(tau):
    B = (1 - math.exp(-kappa * tau)) / kappa
    A = (r_bar - s**2 / (2 * kappa**2)) * (B - tau) - s**2 * B**2 / (4 * kappa)
    return math.exp(A - B * r0)


rng = np.random.default_rng(288)
print("  maturity  formula  steps/yr   left error  trapezium error  left - trapezium")
errors = {}
for tau in (1, 2, 5, 10):
    for per_year in (10, 50, 250):
        n, h = tau * per_year, 1 / per_year
        x = np.full(100_000, r0)
        left = np.zeros(x.size); trap = np.zeros(x.size)
        for _ in range(n):
            new = x + kappa * (r_bar - x) * h + s * math.sqrt(h) *
rng.standard_normal(x.size)
            left += x * h
            trap += 0.5 * (x + new) * h
            x = new
        e_l, e_t = np.exp(-left).mean() - formula(tau), np.exp(-trap).mean() - formula(tau)
        errors[(tau, per_year)] = (e_l, e_t)
        print(f"{tau:10d}  {formula(tau):7.5f}  {per_year:8d}  {e_l:+11.6f}  {e_t:+15.6f}
{e_l - e_t:+16.6f}")
```

What it prints.

```
maturity  formula  steps/yr   left error  trapezium error  left - trapezium
       1  0.96839        10    +0.000134        -0.000060         +0.000194
       1  0.96839        50    +0.000036        -0.000001         +0.000038
       1  0.96839       250    +0.000002        -0.000006         +0.000008
       2  0.93492        10    +0.000166        -0.000130         +0.000296
       2  0.93492        50    +0.000044        -0.000014         +0.000058
       2  0.93492       250    -0.000037        -0.000048         +0.000012
       5  0.83429        10    +0.000185        -0.000192         +0.000377
       5  0.83429        50    -0.000037        -0.000112         +0.000075
       5  0.83429       250    -0.000086        -0.000101         +0.000015
      10  0.68473        10    -0.000162        -0.000497         +0.000334
      10  0.68473        50    -0.000148        -0.000214         +0.000067
      10  0.68473       250    -0.000000        -0.000014         +0.000013
```

Reading it. The errors in the left two columns are mostly sampling noise, about 0.0001 with 100,000 paths. That hides the effect we're after. But the two integrals use the same paths, so their difference, in the last column, has hardly any noise. It's proportional to the step: it falls by 5 from 10 to 50 steps a year, and again by 5 to 250.

Why the left sum is high. The rate starts below its long-run level and tends to rise. A left-endpoint sum uses the lower, earlier values, and so underestimates $\int r\,dt$, which makes the bond too valuable. The size of the effect is about $\frac{1}{2}h(E[r_T] - r_0)P$. With $h = 0.1$, at maturity 10: $E[r_T] = 0.03993$, $P = 0.6847$, so this comes to 3.4e-04. The measured difference is 3.3e-04.

Self-test

1. $g(0.5, 1) = 1 + 0.5 = 1.5$.

2. $g(t, X_t)$ is a martingale (by the tower property), and a martingale's dt term is zero.

3. The term rg: $g_t + ag_x + \frac{1}{2}b^2 g_{xx} = rg$.

SOLUTIONS TO LESSON 29

Finite Differences

Solutions to Exercises 29.1 to 29.8 and the Lesson 29 self-test.

Exercise 29.1

The idea. Subtract two Taylor series: the even terms cancel.

Step 1.

$$f(x+h) - f(x-h) = 2hf'(x) + \tfrac{1}{3}h^3 f'''(x) + \dots$$

Step 2. Divide by $2h$: $\dfrac{f(x+h) - f(x-h)}{2h} = f'(x) + \tfrac{1}{6}h^2 f'''(x) + \dots$

The forward difference keeps the $\frac{1}{2}hf''$ term, so its error is first order in h. The central one's is second order: much smaller, for small h.

The check, with $f = e^x$ at 0 and $h = 0.1$:

- Central: error 0.001668; $\frac{1}{6}h^2 = 0.001667$.
- Forward: error 0.05171; $\frac{1}{2}h = 0.05000$.

Exercise 29.2

The idea. Add the two Taylor series: the odd terms cancel.

$$f(x+h) + f(x-h) = 2f(x) + h^2 f''(x) + \tfrac{1}{12}h^4 f''''(x) + \dots$$

Subtract $2f(x)$, and divide by h^2: $f'' + \frac{1}{12}h^2 f'''' + \dots$

The check. For e^x at 0, $h = 0.1$: error 0.000834; $\frac{1}{12}h^2 = 0.000833$.

Exercise 29.3

The idea. A chain-rule change of variables, then differences, then collect by neighbour.

Step 1. $V(t, S) = U(\tau, y)$, with $y = \ln S$, $\tau = T - t$. So $V_t = -U_\tau$, $V_S = U_y/S$, $V_{SS} = (U_{yy} - U_y)/S^2$.

Step 2. Substitute. $-U_\tau + rU_y + \frac{1}{2}\sigma^2(U_{yy} - U_y) = rU$. With $\nu = r - \frac{1}{2}\sigma^2$:

$$U_\tau = \nu U_y + \tfrac{1}{2}\sigma^2 U_{yy} - rU.$$

Step 3. Discretise. With a forward difference in τ, central differences in y at the old time, and rU_i^{n+1} for the discount:

$$(1 + r\Delta\tau)U_i^{n+1} = U_i^n + \nu\Delta\tau\frac{U_{i+1}^n - U_{i-1}^n}{2\Delta y} + \tfrac{1}{2}\sigma^2\Delta\tau\frac{U_{i+1}^n - 2U_i^n + U_{i-1}^n}{\Delta y^2}.$$

Step 4. Collect.

- U_{i+1}^n: $p_u = \Delta\tau\left(\frac{\sigma^2}{2\Delta y^2} + \frac{\nu}{2\Delta y}\right)$.
- U_{i-1}^n: $p_d = \Delta\tau\left(\frac{\sigma^2}{2\Delta y^2} - \frac{\nu}{2\Delta y}\right)$.
- U_i^n: $p_m = 1 - \frac{\sigma^2\Delta\tau}{\Delta y^2}$.

Step 5. They add to 1. $p_u + p_d = \sigma^2\Delta\tau/\Delta y^2$, which cancels the second term of p_m. ✓

Exercise 29.4

The idea. A wave keeps its shape under this scheme, and only gets multiplied. The multiplier decides everything.

Step 1. With $\nu = r = 0$, $p_u = p_d = c/2$ and $p_m = 1 - c$. Substitute $U_i^n = A^n e^{\mathrm{i}i\xi\Delta y}$, and divide by $A^n e^{\mathrm{i}i\xi\Delta y}$:

$$A = \tfrac{c}{2}e^{\mathrm{i}\xi\Delta y} + 1 - c + \tfrac{c}{2}e^{-\mathrm{i}\xi\Delta y} = 1 - c + c\cos(\xi\Delta y).$$

Step 2. With $1 - \cos\phi = 2\sin^2(\phi/2)$: $A = 1 - 2c\sin^2(\xi\Delta y/2)$.

Step 3. The range. As ξ varies, $\sin^2$ runs from 0 to 1, so A runs from 1 down to $1 - 2c$. For $|A| \le 1$ at every wave, we need $1 - 2c \ge -1$, that is, $c \le 1$.

The worst wave has $\sin^2 = 1$: $\xi\Delta y = \pi$, the zigzag of Lesson 29, with $A = 1 - 2c$.

In words. When $c \le 1$, the new value is a true average of three neighbours, with non-negative weights. An average can't exceed the largest neighbour, so nothing can grow. When $c > 1$, $p_m < 0$: the scheme subtracts the middle value, which amplifies differences, and a zigzag feeds itself.

Exercise 29.5

The idea. Write both work and error in terms of M.

The grid. $\Delta y \propto 1/M$. Stability needs $\Delta\tau \propto \Delta y^2$, so $N \propto M^2$. Work $= MN \propto M^3$. Error $\varepsilon \propto \Delta y^2 \propto M^{-2}$. So $M \propto \varepsilon^{-1/2}$, and work $\propto \varepsilon^{-3/2}$.

Monte Carlo. Error $\propto \text{work}^{-1/2}$, so work $\propto \varepsilon^{-2}$.

Which wins. $\varepsilon^{-3/2}$ grows more slowly than ε^{-2} as $\varepsilon \to 0$. For one share, the grid wins at every useful accuracy, and it wins more as you ask for more. In d dimensions, a grid's work grows as $\varepsilon^{-(d+2)/2}$, so at $d = 2$ the two rates match, and for higher dimensions Monte Carlo wins (Lesson 30).

Exercise 29.6

The idea. A central difference needs a neighbour on each side, and the edge points have only one.

A call. Far below the strike, the option can't realistically finish in the money: $V \approx 0$. Far above, it's sure to be exercised, and so it's a forward: $V \approx S - Ke^{-r\tau}$.

A put. It's the mirror image. Far below, it's sure to be exercised: $V \approx Ke^{-r\tau} - S$. Far above, $V \approx 0$.

How far out. $\ln S_T$ has standard deviation $\sigma\sqrt{T} = 0.2$. A grid to ± 5 standard deviations reaches from about 36.8 to 272. The chance of ending outside it is around 10^{-6}, so any error at the edges has negligible influence at S_0.

Exercise 29.7

European and American puts on the explicit grid, strike on a grid point.

```
import numpy as np


def put_grid(M, N, american):
    dy = 10 * sigma * math.sqrt(T) / M
    gap = math.log(K / S0)
    dy = gap / max(1, round(gap / dy))                        # strike on a grid point
    y = math.log(S0) + dy * (np.arange(M + 1) - M // 2)
    dt = T / N
    a, b = dt * sigma**2 / (2 * dy**2), dt * nu / (2 * dy)
    s = np.exp(y)
    pay = np.maximum(K - s, 0)
    v = pay.copy()
    for n in range(1, N + 1):
        new = v.copy()
        new[1:-1] = ((a + b) * v[2:] + (1 - 2 * a) * v[1:-1] + (a - b) * v[:-2]) / (1 + r *
dt)
        new[0] = K - s[0] if american else K * math.exp(-r * n * dt) - s[0]
        new[-1] = 0.0
        if american:
            new = np.maximum(new, pay)
        v = new
    return float(v[M // 2])


bs_put = o["bs"]["put"]
print("     M       N    European     error    American     error from 9.478")
eu_err = []
for M, N in ((100, 1000), (200, 4000), (400, 16000)):
    e, a = put_grid(M, N, False), put_grid(M, N, True)
    eu_err.append(e - bs_put)
    print(f"{M:6d}  {N:6d}   {e:8.4f}   {e - bs_put:+.5f}   {a:8.4f}   {a - 9.478:+.5f}")
print(f"error ratio, last two grids: {eu_err[1] / eu_err[2]:.2f}")
```

What it prints.

```
     M       N    European     error    American     error from 9.478
   100    1000     9.0117   -0.01318     9.4623   -0.01573
   200    4000     9.0228   -0.00203     9.4750   -0.00300
   400   16000     9.0243   -0.00051     9.4767   -0.00128
error ratio, last two grids: 4.00
```

Reading it. The European error falls by 4.00 between the last two grids: second order in Δy. The American put approaches about 9.477. It's accurate to about the same size, though less regularly, because the early-exercise boundary adds a second kink that moves from step to step.

Exercise 29.8

The three schemes at M = 200, then the change as N doubles at M = 800.

```
bs_call = o["bs"]["call"]


def thomas(lo, di, up, rhs):
    n = rhs.size
    c, d = np.empty(n), np.empty(n)
    c[0], d[0] = up / di, rhs[0] / di
    for i in range(1, n):
        m = di - lo * c[i - 1]
        c[i], d[i] = up / m, (rhs[i] - lo * d[i - 1]) / m
    x = np.empty(n)
    x[-1] = d[-1]
    for i in range(n - 2, -1, -1):
        x[i] = d[i] - c[i] * x[i + 1]
    return x


def scheme(M, N, w):
    # w = 0 explicit, w = 1 implicit, w = 1/2 Crank-Nicolson.
    half = 5 * sigma * math.sqrt(T)
    y = math.log(S0) + np.linspace(-half, half, M + 1)
    dy, dt = y[1] - y[0], T / N
    A, B = sigma**2 / (2 * dy**2), nu / (2 * dy)
    up, md, lo = A + B, -2 * A - r, A - B
    s = np.exp(y)
    v = np.maximum(s - K, 0)
    for n in range(1, N + 1):
        top = s[-1] - K * math.exp(-r * n * dt)
        rhs = v[1:-1] + (1 - w) * dt * (up * v[2:] + md * v[1:-1] + lo * v[:-2])
        rhs[-1] += w * dt * up * top
        v = np.concatenate([[0.0], thomas(-w * dt * lo, 1 - w * dt * md, -w * dt * up, rhs),
[top]])
        if np.abs(v).max() > 1e9:
            return float("nan")
    return float(v[M // 2])


print(f"{'N':>6} {'explicit':>11} {'implicit':>11} {'Crank-Nicolson':>15}   (errors, M =
200)")
for N in (50, 100, 200, 400, 1000):
    ex, im, cn = (scheme(200, N, w) - bs_call for w in (0.0, 1.0, 0.5))
    print(f"{N:6d} {'blown up' if ex != ex else format(ex, '+.5f'):>11} {im:+11.5f}
{cn:+15.5f}")

print("change when N doubles, M = 800 fixed:")
print(f"{'N':>6} {'implicit':>11} {'ratio':>6} {'Crank-Nicolson':>15} {'ratio':>6}")
prev, last = {}, {}
for N in (100, 200, 400, 800):
    row = {}
    for name, w in (("i", 1.0), ("c", 0.5)):
        row[name] = scheme(800, N, w) - scheme(800, 2 * N, w)
    ri = prev["i"] / row["i"] if prev else 0.0
    rc = prev["c"] / row["c"] if prev else 0.0
    show = (lambda x: f"{x:6.2f}") if prev else (lambda x: "      ")
```

```
    print(f"{N:6d} {row['i']:+11.6f} {show(ri)} {row['c']:+15.6f} {show(rc)}")
    prev, last = row, {"i": ri, "c": rc}
```

What it prints.

```
     N    explicit    implicit  Crank-Nicolson   (errors, M = 200)
    50    blown up    -0.01985        -0.00117
   100    blown up    -0.01056        -0.00123
   200    blown up    -0.00591        -0.00125
   400    +0.00509    -0.00358        -0.00125
  1000    -0.00032    -0.00219        -0.00125
change when N doubles, M = 800 fixed:
     N    implicit  ratio  Crank-Nicolson  ratio
   100   -0.004651              +0.000016
   200   -0.002327   2.00       +0.000004   4.00
   400   -0.001164   2.00       +0.000001   4.00
   800   -0.000582   2.00       +0.000000   4.00
```

Reading it. Below $N = 400$, the explicit scheme blows up. Implicit and Crank-Nicolson both give sensible answers. The implicit error shrinks with N, until it reaches the space error; Crank-Nicolson is at that space error from the start, about -0.00125.

The order. To remove the space error from the picture, compare runs at the same M: each change as N doubles comes from time alone. The implicit changes halve (ratio 2.00): **first order**. The Crank-Nicolson changes quarter (ratio 4.00): **second order**.

Self-test

1. $N = \sigma^2 T / \Delta y^2 = 0.04 / 0.0001 = 400$.

2. The forward difference has error proportional to h. The central and second differences have errors proportional to h^2.

3. Solve a tridiagonal system of equations. It's cheap because the Thomas algorithm takes one forward and one backward sweep, about $3M$ operations.

SOLUTIONS TO LESSON 30

Choosing a Method

Solutions to Exercises 30.1 to 30.8 and the Lesson 30 self-test.

Exercise 30.1

The idea. Write cost in terms of error, for each method.

Tree. $\varepsilon \propto 1/n$, so $n \propto 1/\varepsilon$, and cost $\propto n^2 \propto \varepsilon^{-2}$.

Monte Carlo. $\varepsilon \propto 1/\sqrt{M}$, so $M \propto \varepsilon^{-2}$, and cost $\propto \varepsilon^{-2}$.

So on one share they grow at the same rate. What decides between them is the constant, and it heavily favours the tree: 125,250 nodes gave an error of 0.0021 in Lesson 30, against a standard error of 0.0126 from 1,000,000 paths. And a tree's steady error can be extrapolated, which improves its rate. (An explicit grid does better still: $\varepsilon^{-3/2}$.)

With more assets. A d-asset tree has cost $\propto n^{d+1}$, so $\propto \varepsilon^{-(d+1)}$, which grows terribly. Monte Carlo stays at ε^{-2}. Beyond a few assets, it has no competition.

Exercise 30.2

The idea. Points multiply by 100 for each asset.

A grid with 100 points per asset, at 8 bytes a point, for one time step.

ASSETS	POINTS	GIGABYTES
1	1.0e+02	8.0e-07
2	1.0e+04	8.0e-05
3	1.0e+06	8.0e-03
5	1.0e+10	8.0e+01
10	1.0e+20	8.0e+11

Time steps. With 100 points in each direction across $\pm 5\sigma\sqrt{T}$, $\Delta y = 10\sigma\sqrt{T}/100 = 0.02$. The explicit limit, $\sigma^2 \Delta\tau/\Delta y^2 \leq 1$ for one asset, gives $\Delta\tau \leq 0.01$, or $N \geq 100$. For d assets, the limit is about d times tighter, because each direction adds its own term. So

for $d = 3$, you need about 300 steps, and for $d = 5$, about 500.

On a laptop. With perhaps 16 gigabytes of memory, $d = 4$ fits (0.8 gigabytes) and $d = 5$ doesn't (80 gigabytes). With 10^{10} points per step at $d = 5$, the time would be prohibitive anyway. So a grid is out at about $d = 4$ or $d = 5$. In practice, it's already uncomfortable at 3.

Exercise 30.3

The idea. Count dimensions, then look for the difficulties: early exercise, path dependence, jumps in the payoff.

- **A European put:** the formula. It's exact and instant.
- **An American put on one share:** a tree or a grid. One dimension, and early exercise takes one line.
- **A call on the average of 20 shares:** Monte Carlo. Twenty dimensions rule out a grid. Use the geometric average or the moment-matched Black-Scholes value as a control variate.
- **A one-share barrier option that dies at 80:** a grid, with a boundary placed exactly at 80. (A formula exists for continuous monitoring. Monte Carlo works, but must correct for crossings between steps.)
- **An American option on 5 shares:** Longstaff-Schwartz. A grid is out, and early exercise has to be estimated.
- **The delta of a digital on one share:** the formula if it's available. On paths, the likelihood ratio method. On a tree, neighbouring nodes work, but the payoff's jump makes them noisy near the strike.

Exercise 30.4

The idea. Treat the path as a smooth function of S_0, with Z fixed.

Step 1. $S_T = S_0 G$, with $G = e^{(r - \frac{1}{2}\sigma^2)T + \sigma\sqrt{T}Z}$. So $\partial S_T / \partial S_0 = G = S_T / S_0$.

Step 2. The derivative of $\max(S_T - K, 0)$ with respect to S_T is $\mathbf{1}\{S_T > K\}$. By the chain rule:

$$\frac{\partial}{\partial S_0}\left[e^{-rT}\max(S_T - K, 0)\right] = e^{-rT}\mathbf{1}\{S_T > K\}\frac{S_T}{S_0}.$$

Step 3. Its average. $\tilde{E}[e^{-rT}S_T\mathbf{1}\{S_T > K\}]/S_0 = S_0 N(d_+)/S_0 = N(d_+)$, using the share term of Lesson 20.

Why the kink is fine. The payoff is continuous, and only its derivative jumps, at one point, which has zero probability. Its derivative with respect to S_0 exists on almost every path, and the slope is bounded. So the average of the derivative is the derivative of the average.

Exercise 30.5

The idea. The derivative can go on the density instead of the payoff.

Why pathwise fails. A digital's payoff is flat on each side of K, so along almost every path, its derivative is 0. The average of those is 0, while the true delta isn't. All the sensitivity is at the jump, and it's invisible to a path-by-path derivative. Average and derivative can't be swapped here.

The likelihood ratio.

Step 1. Write $V = e^{-rT}\int f(s)\,p(s;S_0)\,ds$, where p is the lognormal density of S_T. The only dependence on S_0 is in p.

Step 2. Differentiate under the integral, and multiply and divide by p:

$$\frac{\partial V}{\partial S_0} = e^{-rT}\int f(s)\frac{\partial_{S_0} p}{p}p\,ds = e^{-rT}\tilde{E}\Big[f(S_T)\frac{\partial \ln p}{\partial S_0}\Big].$$

Step 3. The weight. $\ln p = -\dfrac{(\ln s - \ln S_0 - (r-\frac{1}{2}\sigma^2)T)^2}{2\sigma^2 T} + (\text{terms without } S_0)$. Differentiating gives

$$\frac{\partial \ln p}{\partial S_0} = \frac{\ln s - \ln S_0 - (r-\frac{1}{2}\sigma^2)T}{\sigma^2 T\,S_0} = \frac{\sigma\sqrt{T}Z}{\sigma^2 T S_0} = \frac{Z}{S_0\sigma\sqrt{T}}.$$

So the delta estimate is the payoff times $Z/(S_0\sigma\sqrt{T})$. It needs no derivative of f, so any payoff is fine.

Exercise 30.6

The idea. A rule you can actually follow can't beat the best rule.

The low estimate. The true American value is the supremum over all exercise rules that don't look into the future. A fitted rule, once fixed, is one such rule. Applied to fresh paths, independent of the fit, the rule doesn't look into their future, and its average is an unbiased estimate of that rule's value. That value can't exceed the supremum. So on average, the estimate is at or below the true value.

Why reusing paths can push it up. On the fitting paths, the regression has seen each path's own future cash flow. So the rule is partly tailored to that path's future, a small amount of foresight. Exercise decisions benefit from it, and that's a high bias. It's usually small, as the regression has few parameters and many paths, and it partly offsets the low bias from an imperfect rule.

Exercise 30.7

Longstaff-Schwartz with separate paths for fitting and valuing.

```
import numpy as np


def simulate(paths, dates, gen):
    h = T / dates
    return S0 * np.exp(((r - 0.5 * sigma**2) * h + sigma * math.sqrt(h) *
gen.standard_normal((paths, dates))).cumsum(axis=1))


def fit_rule(paths_s, dates, degree):
    # Work backwards on the fitting paths; keep one polynomial per exercise date.
    df = math.exp(-r * T / dates)
    cash = np.maximum(K - paths_s[:, -1], 0.0)
    coefs = [None] * dates
    for i in range(dates - 2, -1, -1):
        cash *= df
        itm = K - paths_s[:, i] > 0
        x = paths_s[itm, i] / K
        c = np.polyfit(x, cash[itm], degree)
        coefs[i] = c
        ex = K - paths_s[itm, i]
        do = ex > np.polyval(c, x)
        cash[np.where(itm)[0][do]] = ex[do]
    return coefs


def value(paths_s, dates, coefs):
    # Follow the fitted rule on fresh paths: exercise the first time it says so.
    df = math.exp(-r * T / dates)
    n_p = paths_s.shape[0]
    pv = np.zeros(n_p)
    alive = np.ones(n_p, bool)
    for i in range(dates - 1):
        s = paths_s[:, i]
        go = alive & (K - s > 0)
        go[go] = (K - s[go]) > np.polyval(coefs[i], s[go] / K)
        pv[go] = (K - s[go]) * df ** (i + 1)
        alive &= ~go
    pv[alive] = np.maximum(K - paths_s[alive, -1], 0) * df**dates
    return pv.mean(), pv.std(ddof=1) / math.sqrt(n_p)


tree = o["am"]["tree"]
print(f"tree, exercise at any time: {tree:.4f}")
print("degree  dates   value      se     gap from the tree")
res = {}
for degree in (1, 2, 3, 5):
    for dates in (12, 50):
        gen = np.random.default_rng(307 + degree)
        coefs = fit_rule(simulate(100_000, dates, gen), dates, degree)
        v, se = value(simulate(200_000, dates, gen), dates, coefs)
        res[(degree, dates)] = v
        print(f"{degree:6d}  {dates:5d}   {v:7.4f}  {se:.4f}   {v - tree:+.4f}")
```

What it prints.

```
tree, exercise at any time: 9.4776
degree  dates   value      se     gap from the tree
     1     12    9.3529  0.0215   -0.1247
     1     50    9.3406  0.0207   -0.1370
     2     12    9.4100  0.0206   -0.0675
     2     50    9.4322  0.0199   -0.0453
     3     12    9.4434  0.0206   -0.0341
     3     50    9.4657  0.0200   -0.0119
     5     12    9.4142  0.0203   -0.0633
     5     50    9.4547  0.0201   -0.0229
```

Reading it. All values lie below the tree, as Exercise 30.6 says they should, apart from sampling noise. The degree of the polynomial matters most: a straight line is clearly too crude (9.341 at 50 dates), while degree 3 gets to 9.466. Going up to 5 doesn't help (9.455): the extra terms fit noise. More exercise dates help when the rule is good: for degree 3, from 9.443 at 12 dates to 9.466 at 50.

Exercise 30.8

The digital's delta, by bumping and by the likelihood ratio.

```
dm = (math.log(S0 / K) + (r - 0.5 * sigma**2) * T) / (sigma * math.sqrt(T))
disc = math.exp(-r * T)
exact = disc * phi(dm) / (S0 * sigma * math.sqrt(T))
rng = np.random.default_rng(308)
M = 1_000_000
Z = rng.standard_normal(M)
grow = np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * Z)
print(f"exact delta of the cash digital: {exact:.5f}")
print("    bump    estimate      se     paths that differ")
out_se = {}
for b in (5.0, 1.0, 0.1, 0.01):
    d = disc * (((S0 + b) * grow > K).astype(float) - ((S0 - b) * grow > K).astype(float)) /
(2 * b)
    out_se[b] = d.std() / 1000
    print(f"{b:8.2f}   {d.mean():8.5f}  {out_se[b]:.5f}   {(d != 0).sum():8d}")
lr = disc * (S0 * grow > K) * Z / (S0 * sigma * math.sqrt(T))
print(f"likelihood ratio: {lr.mean():.5f}  se {lr.std() / 1000:.5f}")
```

What it prints.

```
exact delta of the cash digital: 0.01900
    bump    estimate      se     paths that differ
    5.00    0.01874  0.00004     193133
    1.00    0.01901  0.00009      39188
    0.10    0.01904  0.00030       3925
    0.01    0.01868  0.00095        385
likelihood ratio: 0.01903  se 0.00003
```

Reading it. Bumping faces a dilemma. A large bump is biased: at 5, the difference quotient averages the delta over a wide range. A small bump is noisy: at 0.01, only a few hundred paths land in the tiny window between the two digitals, and each contributes a huge amount, $1/(2b)$. The standard error grows like $1/\sqrt{b}$: 0.00009 at 1, 0.00095 at 0.01.

The likelihood ratio has no bump, no bias, and a standard error of 0.00003, against an exact value of 0.01900. That is the one to use.

Self-test

1. It needs about $100^{10} = 10^{20}$ points. The memory alone doesn't exist.

2. Because the price difference is tiny compared with the noise in either estimate. With the same random numbers, nearly all the noise cancels in the subtraction. With different ones, it adds.

3. The likelihood ratio method. The pathwise method gives 0 on almost every path, because the payoff is flat except at the jump.